

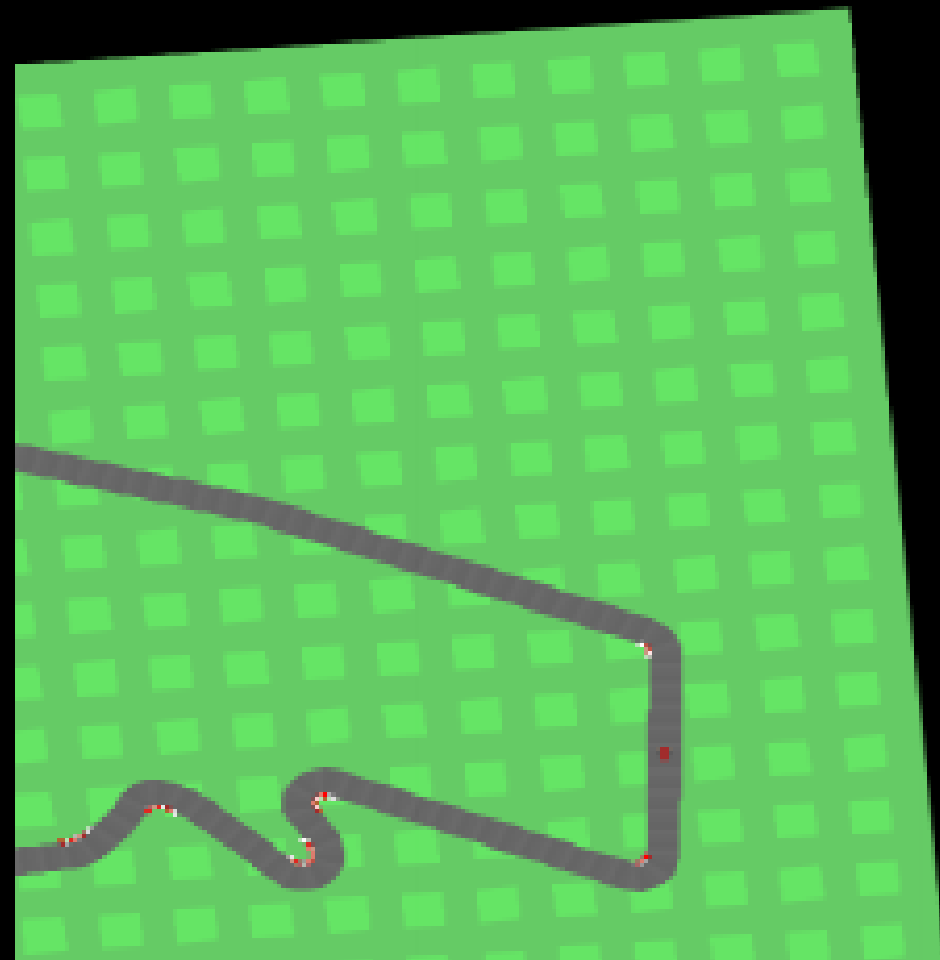
LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

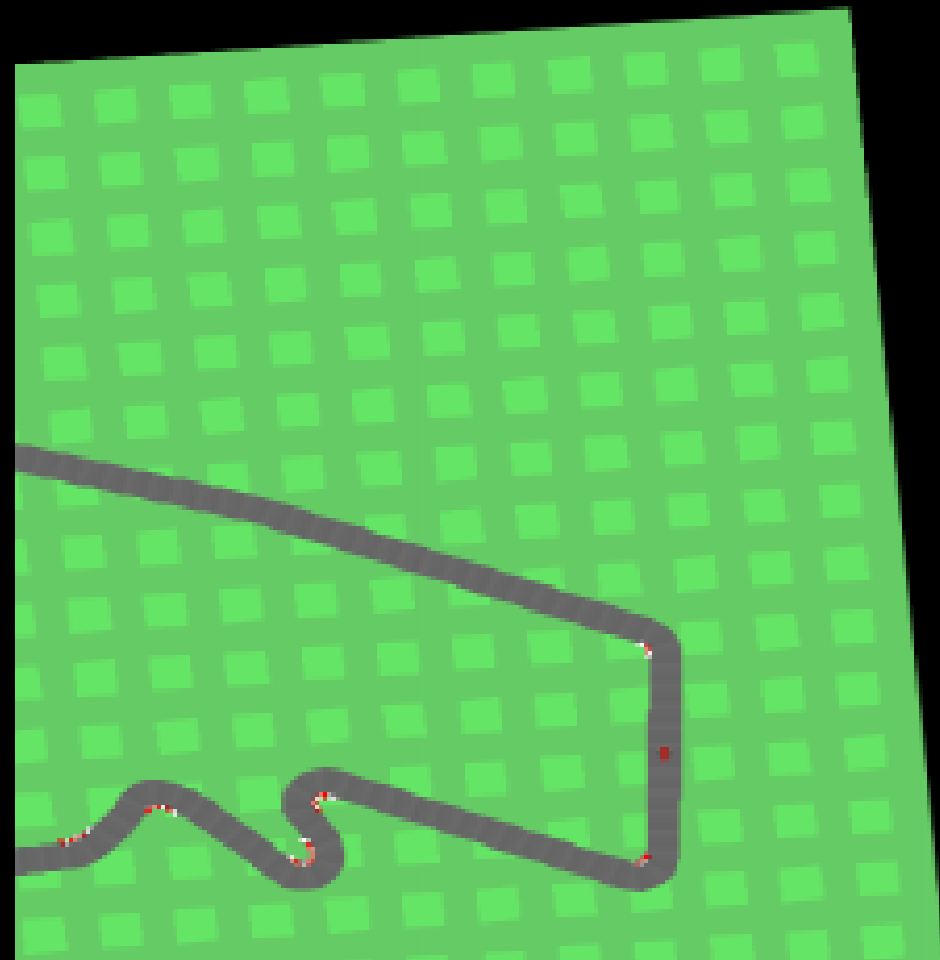
목차 03 다양한 강화학습 알고리즘 적용

1. Q-Learning의 한계와 DQN
2. 정책 함수 근사
3. Actor-Critic

1. Q-Learning의 한계와 DQN

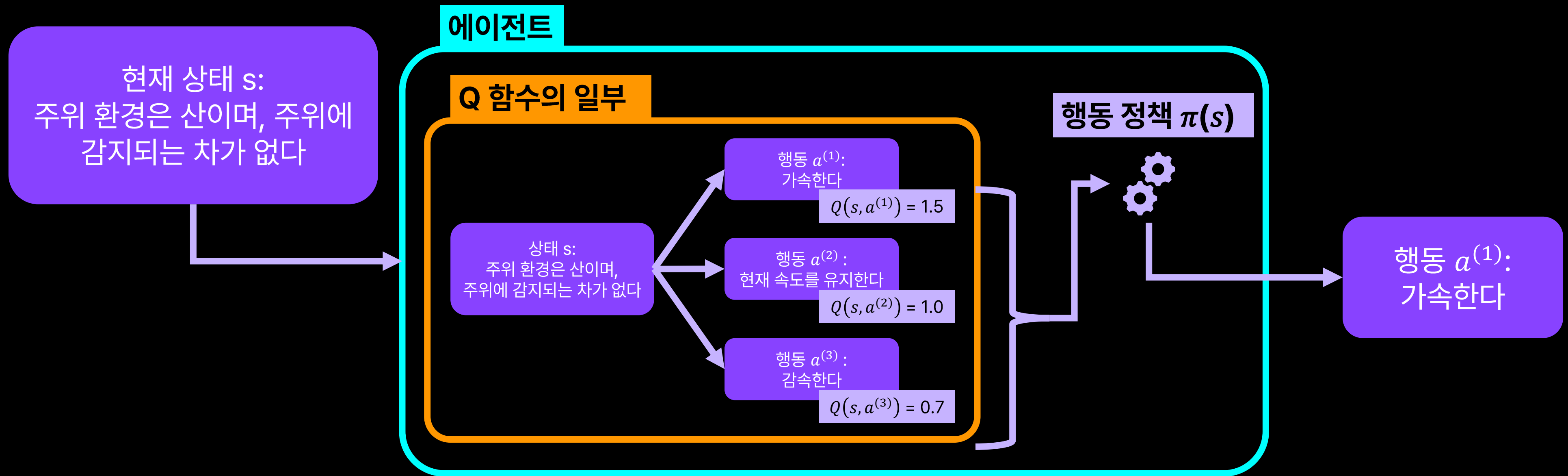


- Gymnasium Car Racing 환경
- 랜덤한 트랙을 최대한 빠르게 완주하는 것이 목표
- State: 자동차와 트랙을 위에서 바라본 96 x 96 크기 RGB 이미지
 - 한 State의 크기: (화면 크기) x (RGB) x (가능한 픽셀 값) = $(96 \times 96) \times 3 \times 256 = 7,077,888$
- Action: 다섯 행동 중 하나를 선택
 - {조작 안함, 왼쪽으로 틀기, 오른쪽으로 틀기, 가속, 감속}
- Reward – 매 프레임마다 제공됨
 - 매 프레임마다 -0.1
 - 트랙을 벗어날 경우: -100
 - 트랙 타일 N개 중 통과한 트랙 만큼 $1000/N$ (=완주하면 1000)



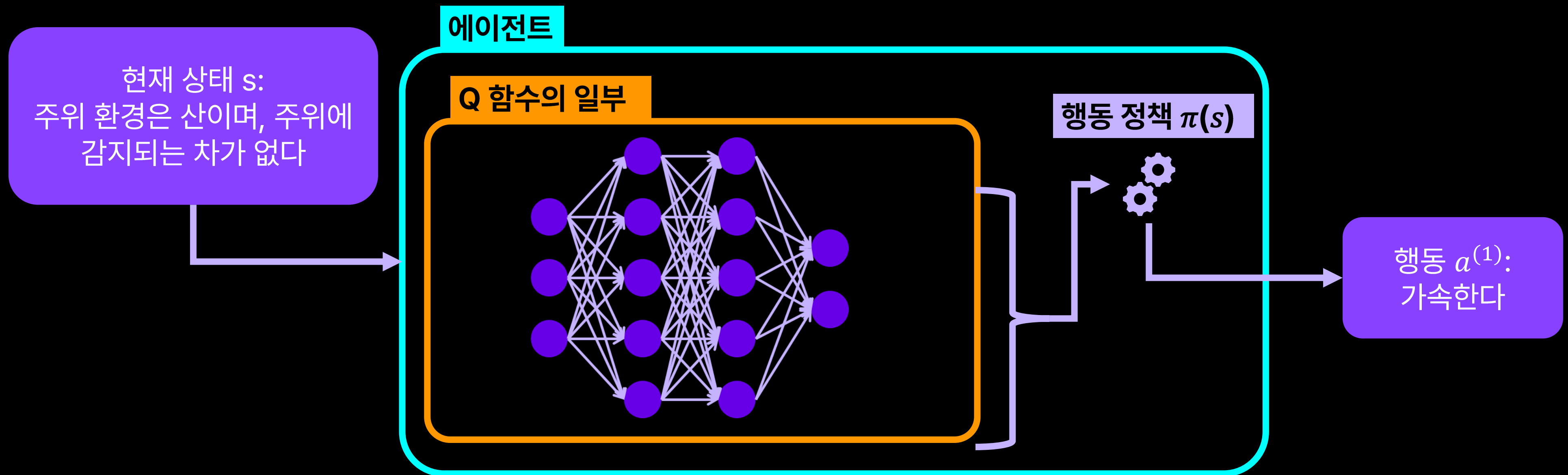
- Car Racing 환경의 State space의 크기: 1.04×10^{66583}
 - (픽셀 값) ^ ((화면 크기) x (RGB))
- 각 State에 5개 행동에 대한 Q-value를 저장하기 위해 8 비트를 사용한다고 하면, 약 1.04×10^{66583} 바이트의 저장 공간이 필요하다.
- 자율주행 AI는 훨씬 많은 경우의 수에 대한 판단을 내려야 한다.
- 현실적으로 모든 $Q(s, a)$ 경우의 수에 대한 Q 함수의 값을 계산하는 것은 불가능하다.

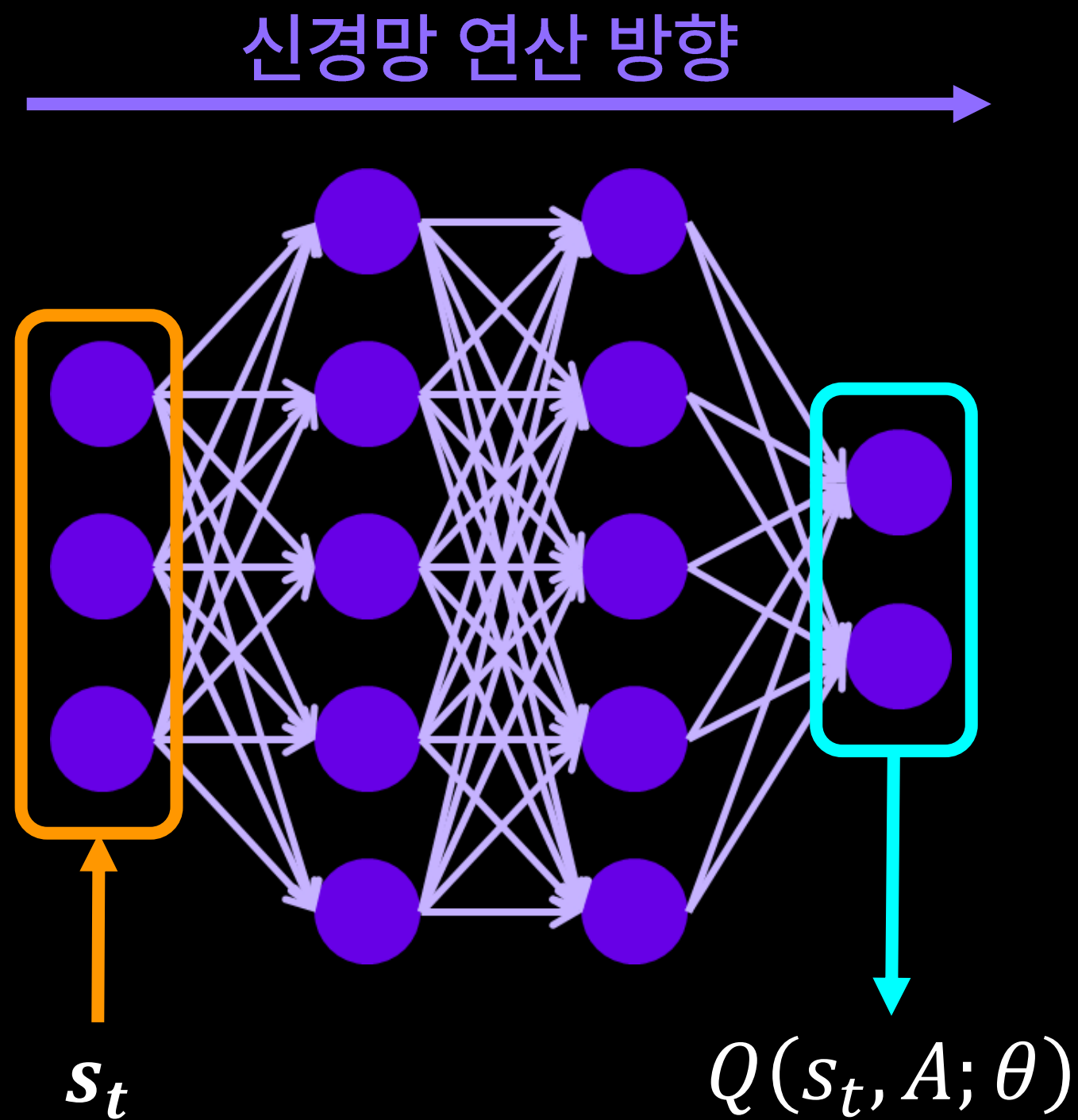
- 모든 상태에 대한 행동의 가치를 나타내는 **Q 함수를 인공신경망으로 근사**하여 표현한다.
- 이를 통해 실제 Q 함수 대비 현실적인 크기로 Q 함수를 나타낼 수 있다.





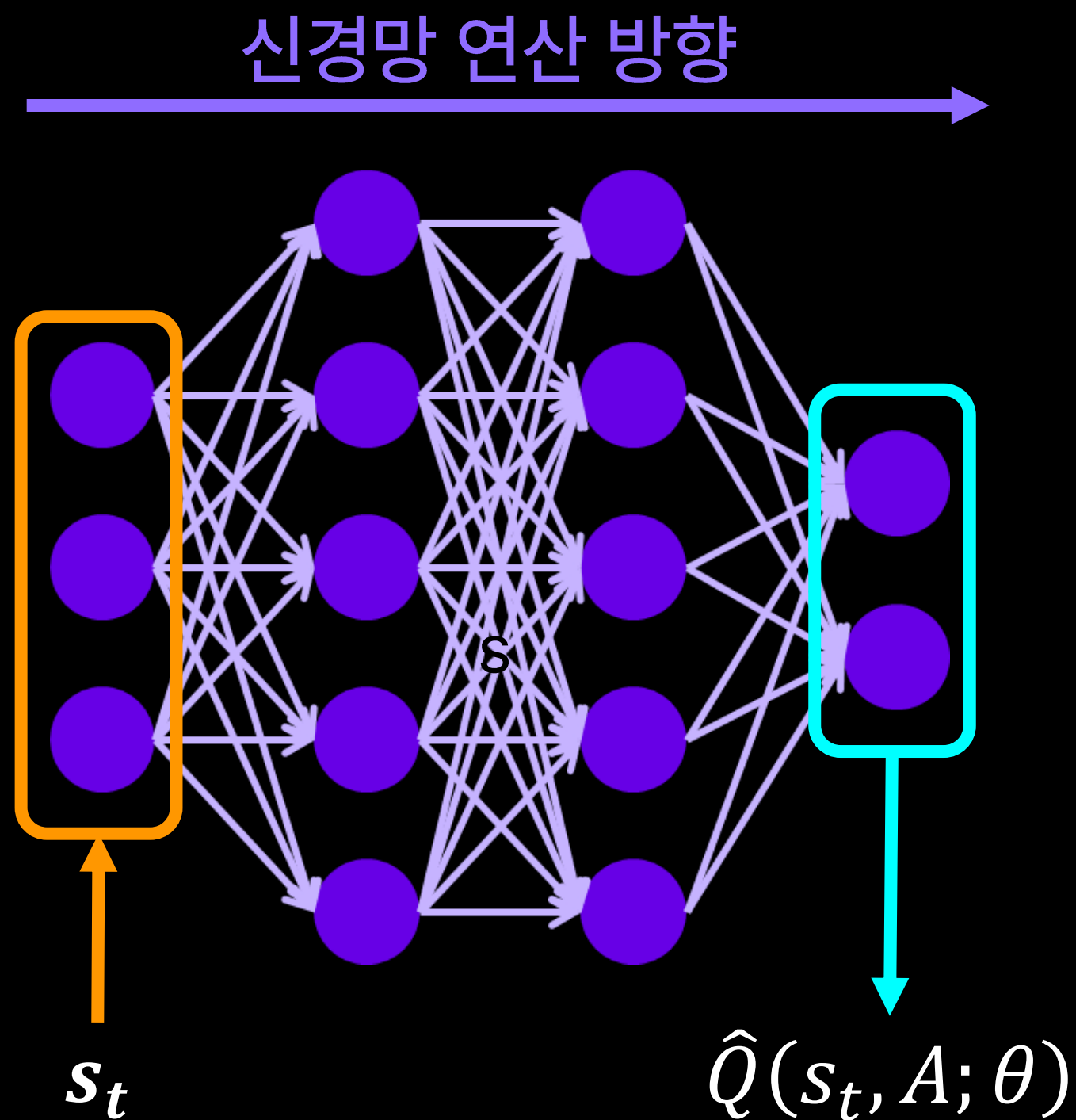
- 모든 상태에 대한 행동의 가치를 나타내는 **Q 함수를 인공신경망으로 근사**하여 표현한다.
- 이를 통해 실제 Q 함수 대비 현실적인 크기로 Q 함수를 나타낼 수 있다.





[이산적 행동 공간에서의 인공신경망 Q 함수]

- 인공신경망을 통해 Q 함수를 근사하여 표현
 - $Q(S, A; \theta) \approx Q(S, A)$
 - θ 는 파라미터를 나타내며, 여기서는 인공신경망을 의미
- 입력으로 현재 State를 수학적으로 나타낸 값을 입력
 - Vector 형태로 입력
- 출력은 각 Action에 대한 Q 값으로 해석
- 신경망의 학습 목표는 이상적인 Q 함수의 값 $Q^*(S, A)$ 과 근사한 함수의 값 $Q(S, A; \theta)$ 간의 차이를 최소화 하는 것이다.



[이산적 행동 공간에서의 인공신경망 Q 함수]

- 신경망의 학습 목표는 이상적인 Q 함수의 값 $Q^*(S, A)$ 과 근사한 함수의 값 $Q(S, A; \theta)$ 간의 차이를 최소화 하는 것이다.
- 그러나, 이상적인 Q 함수 $Q^*(S, A)$ 의 값을 알 수는 없으므로, Q-Learning의 오차와 유사한 목표 함수를 대신 사용한다.

$$\text{오차} = \underbrace{R_t + \gamma \max_a Q(s_{t+1}, a)}_{\text{실제 Q 함수 값}} - \underbrace{Q(s_t, A_t)}_{\text{예측 Q 함수 값}}$$

Q-Learning의 오차

- 시간 t에서의 목표 함수 $J_t(\theta) =$

$$\mathbb{E}[\underbrace{(R_t + \gamma \max_a Q(s_{t+1}, a; \theta))}_{\text{목표 Q 함수 값}} - \underbrace{Q(s_t, a_t; \theta)}_{\text{예측 Q 함수 값}})^2]$$



1. 관측 값 간에 시간적 상관 관계가 존재한다.
2. 예측 Q 함수 값 $Q(s_t, a_t; \theta)$ 와, 목표 Q 함수 값 $R_t + \gamma \max_a Q(s_{t+1}, a; \theta)$ 이 같은 신경망 θ 를 공유한다.



1. 관측 값 간에 시간적 상관 관계가 존재한다.

- (s_{t+1}, a_{t+1}) 이 관측될 가능성은 이전 상태 및 행동인 (s_t, a_t) 에 크게 의존한다.
- 딥러닝은 데이터가 같은 분포 하에서 서로 독립적(i.i.d.) 이라 가정한다. 만약 이 가정이 깨진다면 학습이 불안정해질 수 있다.

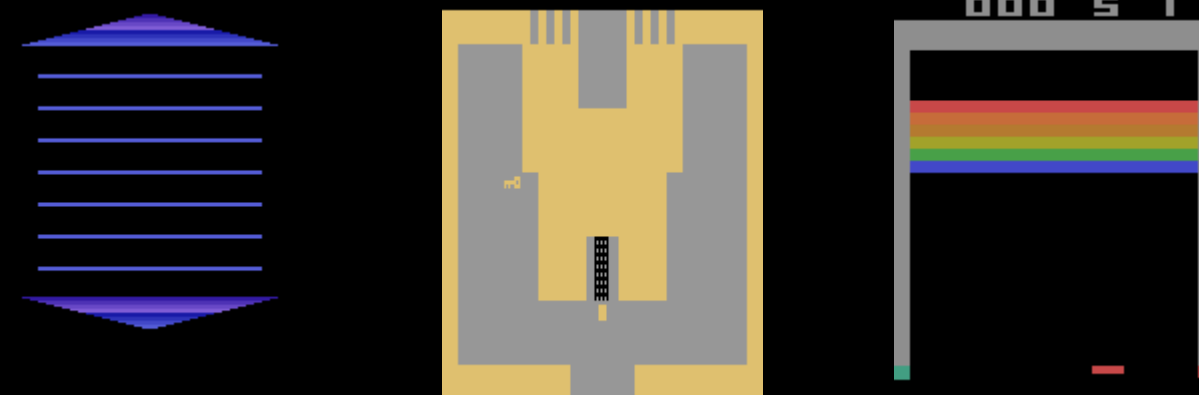


1. 예측 Q 함수 값 $Q(s_t, a_t; \theta)$ 와, 목표 Q 함수 값 $R_t + \gamma \max_a Q(s_{t+1}, a; \theta)$ 이 같은 신경망 θ 를 공유한다.
 - 예측 값과 목표 값이 같은 신경망 θ 을 공유하기 때문에, 학습 목표가 계속 변경되어 학습이 불안정해진다.

$$J_t(\theta) = \mathbb{E}[\underbrace{(R_t + \gamma \max_a Q(s_{t+1}, a; \theta) - Q(s_t, a_t; \theta))}_{\substack{\text{목표} \\ \text{Q 함수 값}}}]^2]$$



- Google DeepMind에서 2013년과 2015년에 발표한 강화학습 모델
 - 2013년에는 NeurIPS에, 2015년에는 Nature지에 출판
- Atari 게임의 현재 이미지를 입력으로 받고, Atari 게임기의 버튼을 Action space로 설정하여 최대한 높은 점수를 얻도록 학습



[Atari 게임의 예시]



- 관측 값 간에 **시간적 상관 관계**를 완화하기 위한 구조 (문제 #1)

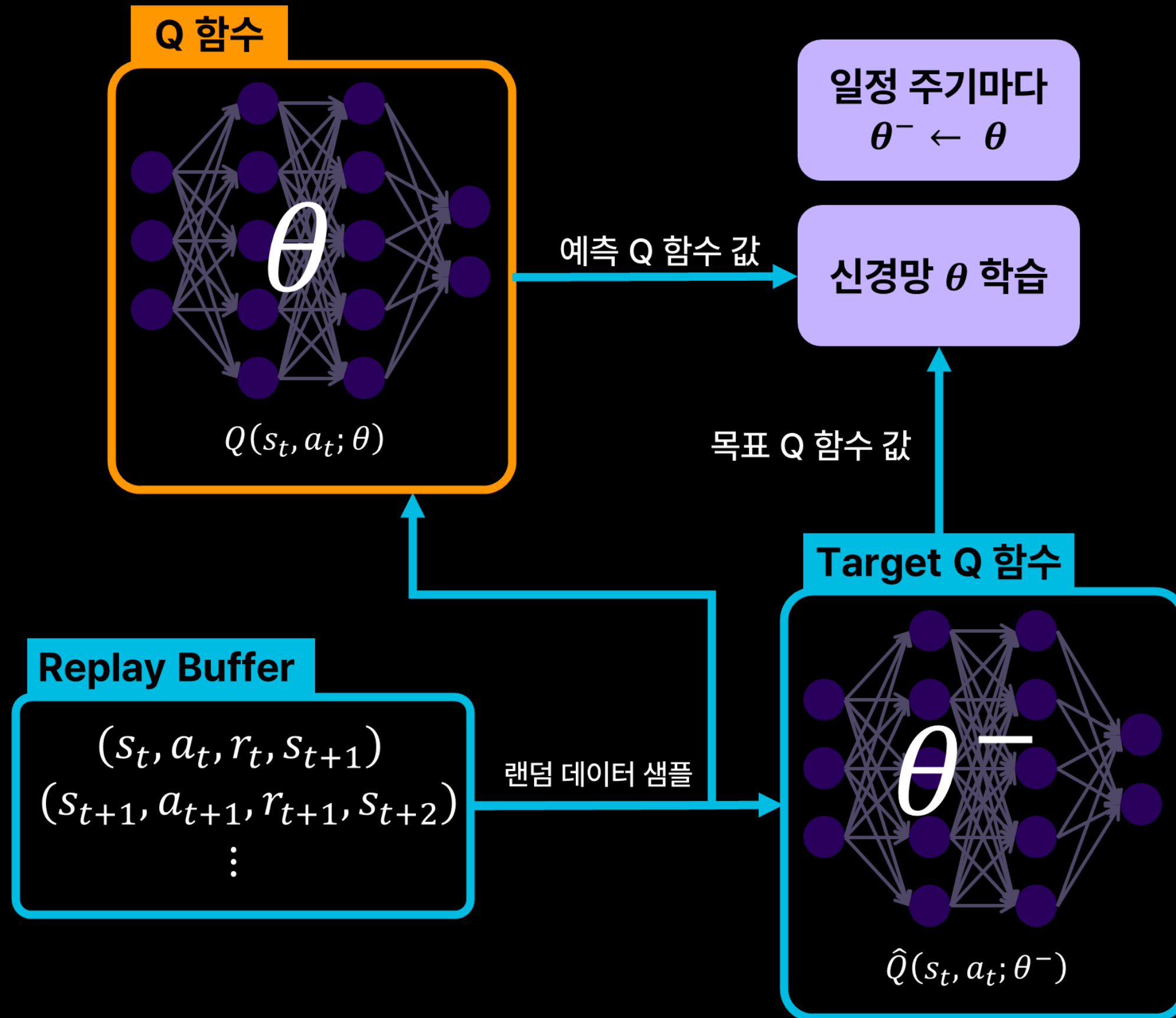
Replay Buffer

(s_t, a_t, r_t, s_{t+1})
 $(s_{t+1}, a_{t+1}, r_{t+1}, s_{t+2})$
 $\vdots$

- (s_t, a_t, r_t, s_{t+1}) 데이터를 저장하고, 충분한 데이터가 모이면 그 중 D개의 데이터를 **랜덤하게** 뽑아서 학습
- 데이터를 입력하는 순서를 섞어서 시간적 상관 관계를 약화시킨다.



Fixed Target



- 예측 Q 함수 값 $Q(s_t, a_t; \theta)$ 와 목표 Q 함수 값 $R_t + \gamma \max_a Q(s_{t+1}, a; \theta)$ 이 같은 신경망을 공유하는 문제를 해결하기 위한 구조 (문제 #2)
- 목표 Q 함수에 사용하는 신경망 θ^- 를 예측 Q 함수의 신경망 θ 과 분리함으로써 Target이 계속 움직이는 문제를 완화한다.
- 일정 주기마다 신경망 θ^- 의 값을 신경망 θ 의 값으로 업데이트 한다.

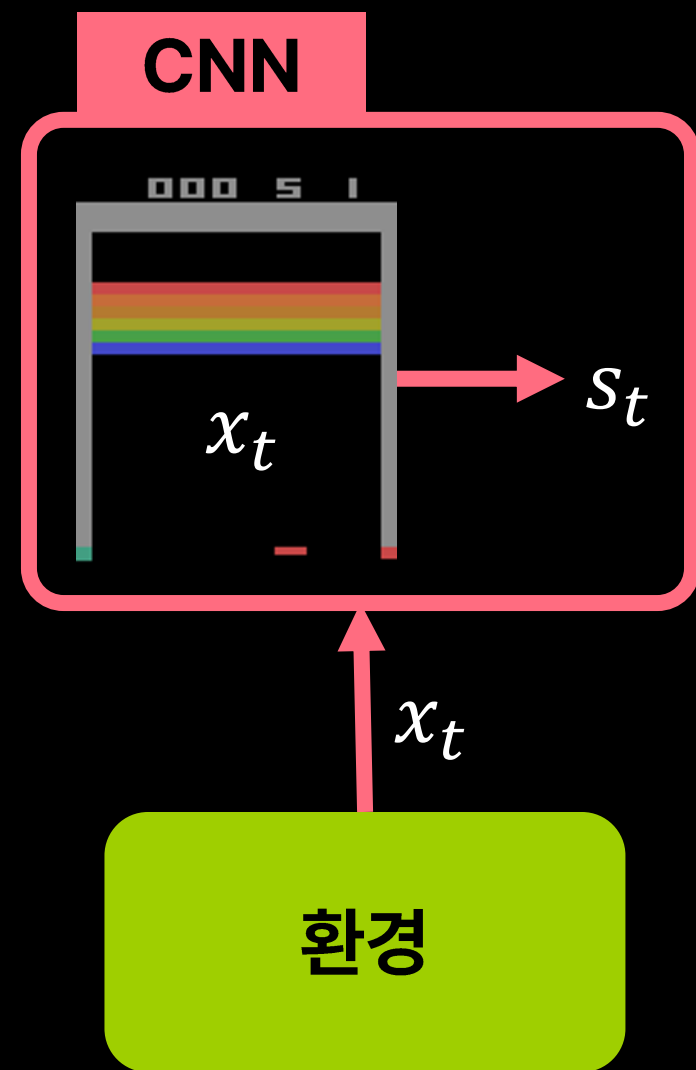


Deep Q-Network

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

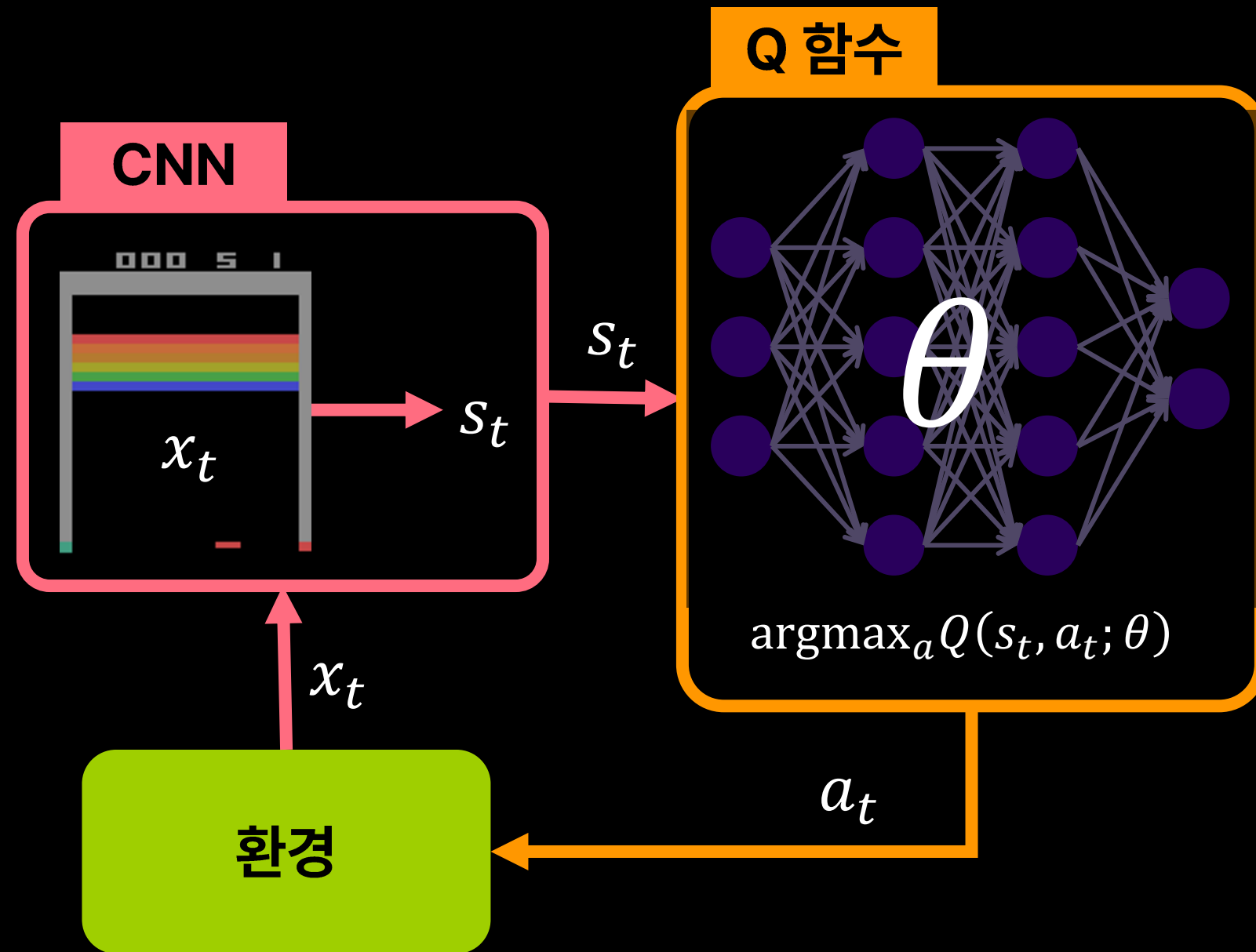
1. Q-Learning의 한계와 DQN



1. 화면 이미지 x_t 를 입력 받아 CNN을 통해 상태 벡터 s_t 로 변환한다.



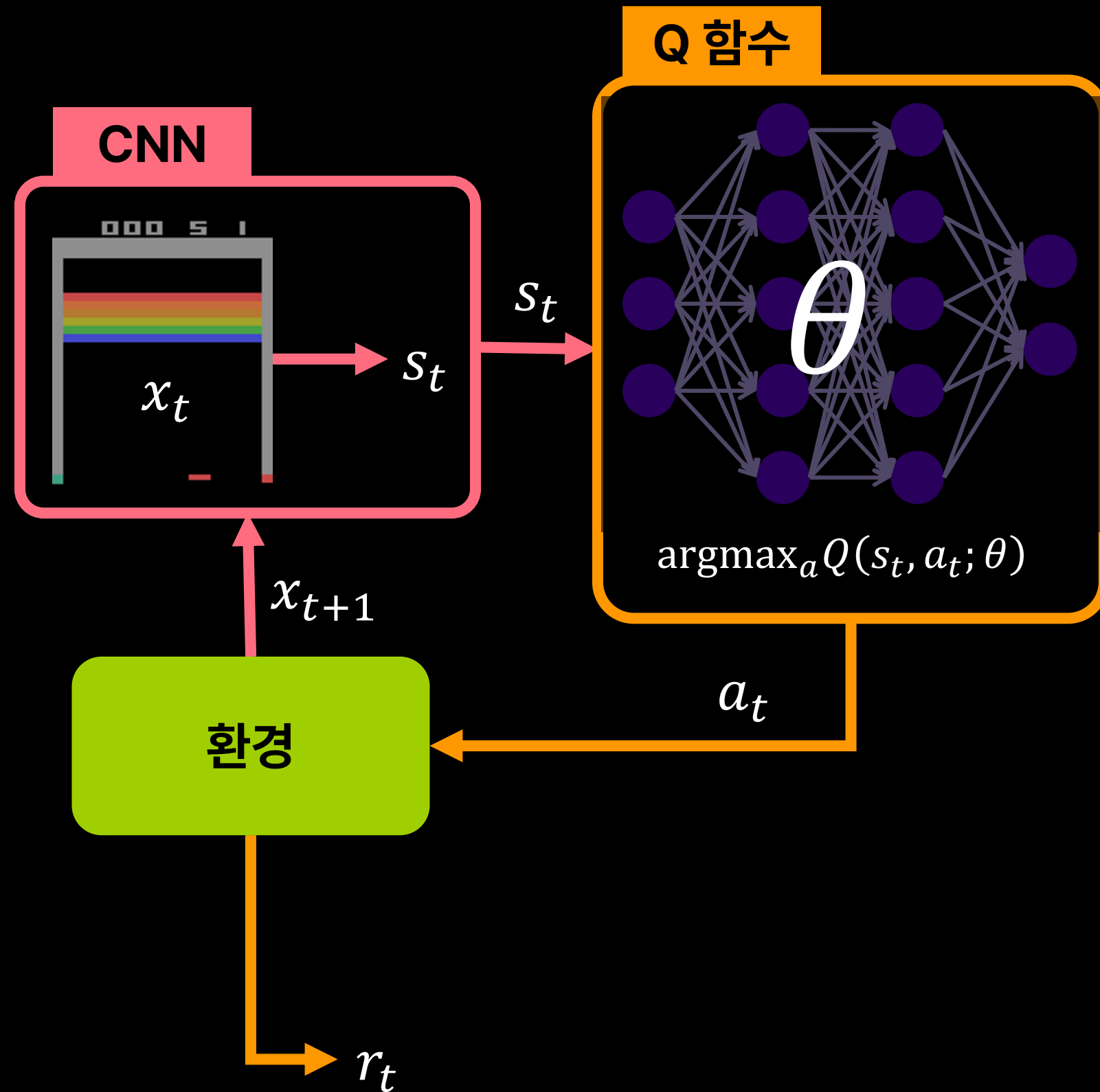
Deep Q-Network



1. 화면 이미지 x_t 를 입력 받아 CNN을 통해 상태 벡터 s_t 로 변환한다.
2. Q 함수에 상태 벡터 s_t 를 입력하고, 생성한 행동 a_t 를 환경에 입력한다.



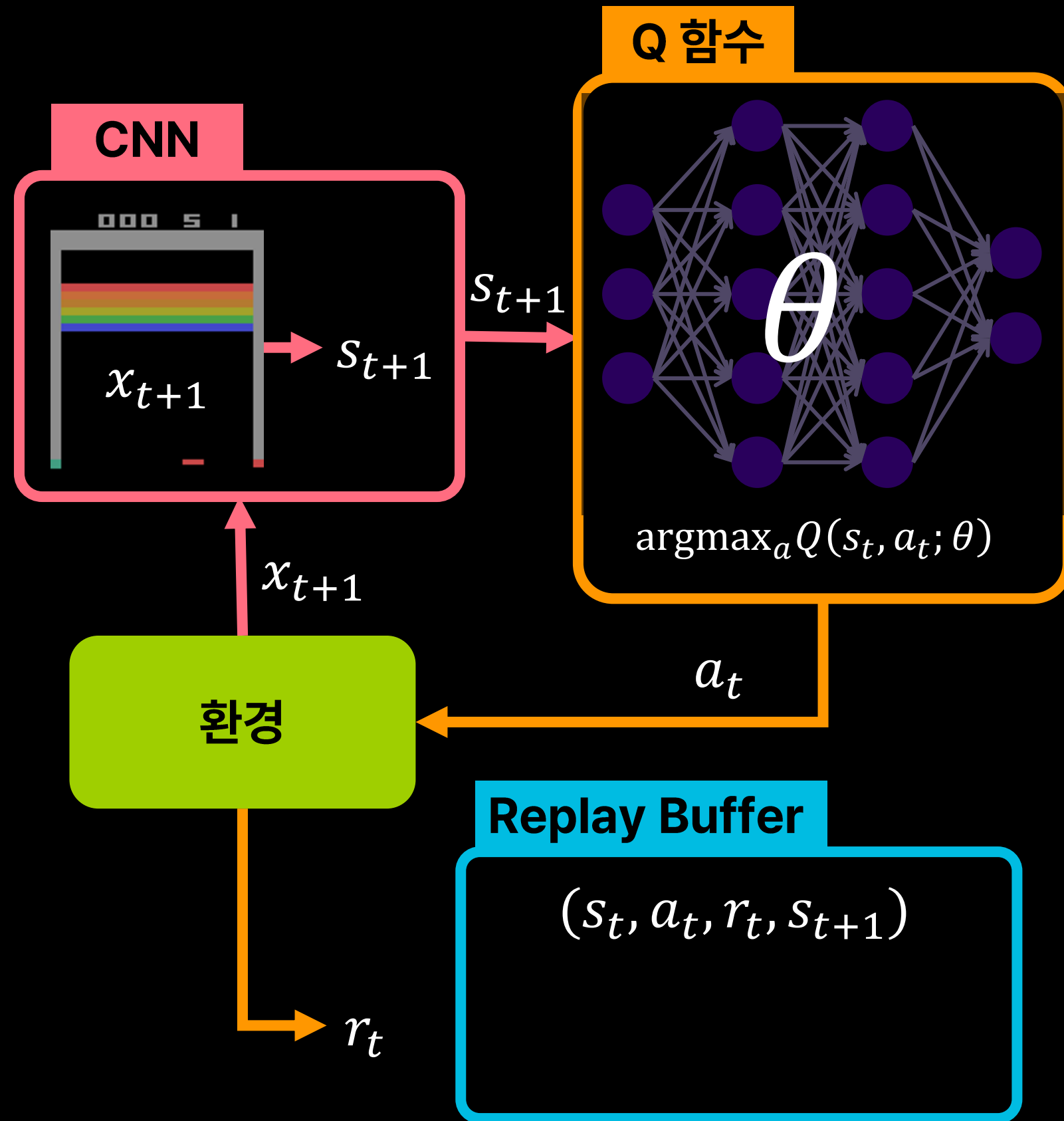
Deep Q-Network



1. 화면 이미지 x_t 를 입력 받아 CNN을 통해 상태 벡터 s_t 로 변환한다.
2. Q 함수에 상태 벡터 s_t 를 입력하고, 생성한 행동 a_t 를 환경에 입력한다.
3. 환경이 다음 화면 x_{t+1} 과 보상 r_t 를 반환한다.



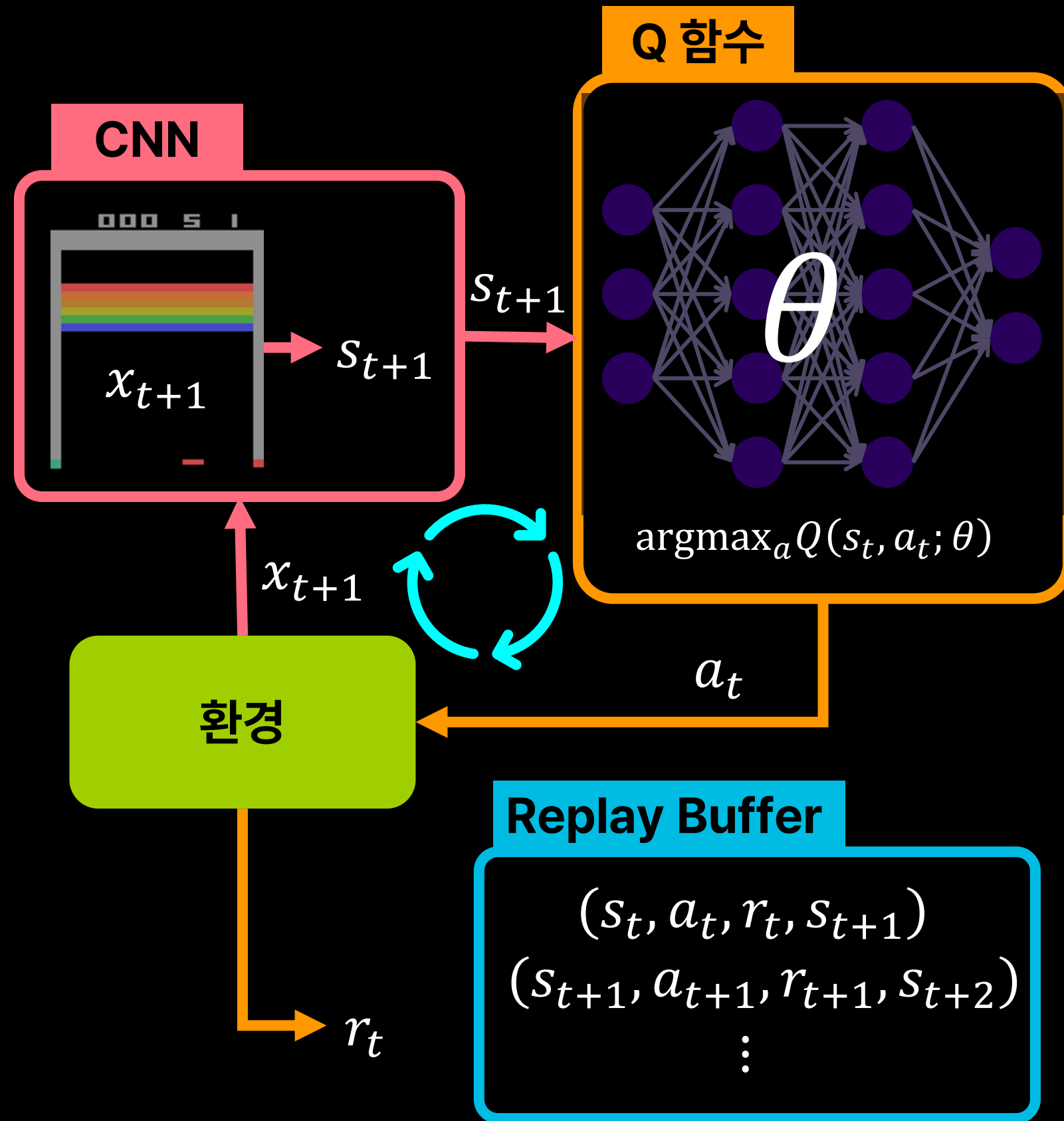
Deep Q-Network



- 반환된 x_{t+1} 를 상태 벡터 s_{t+1} 으로 변환하고,
 (s_t, a_t, r_t, s_{t+1}) 을 Replay Buffer에 저장한다.



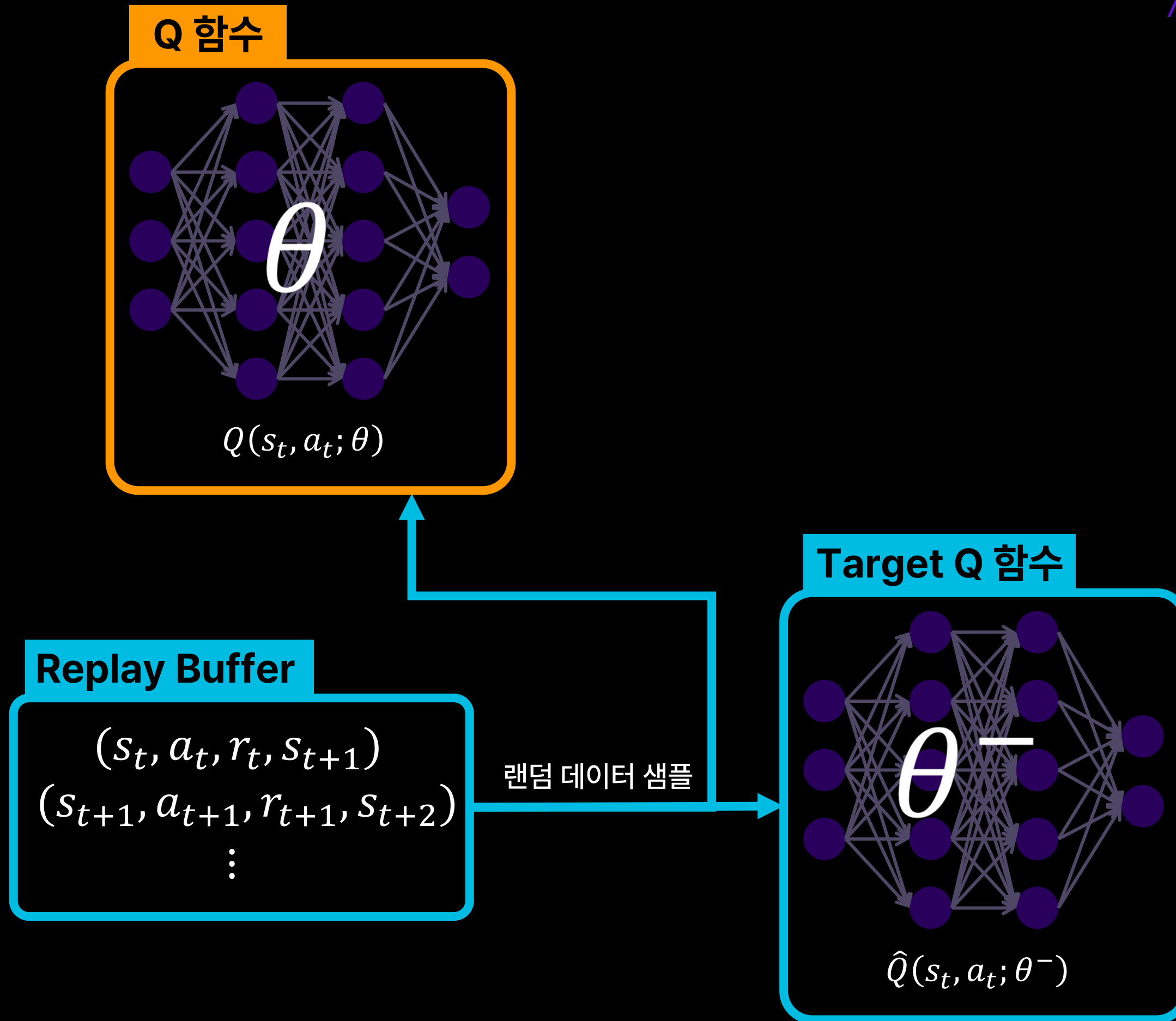
Deep Q-Network



- 반환된 x_{t+1} 를 상태 벡터 s_{t+1} 으로 변환하고, (s_t, a_t, r_t, s_{t+1}) 을 Replay Buffer에 저장한다.
- Replay Buffer에 충분한 데이터가 없다면, 충분한 데이터가 모일 때 까지 1~4를 반복한다.



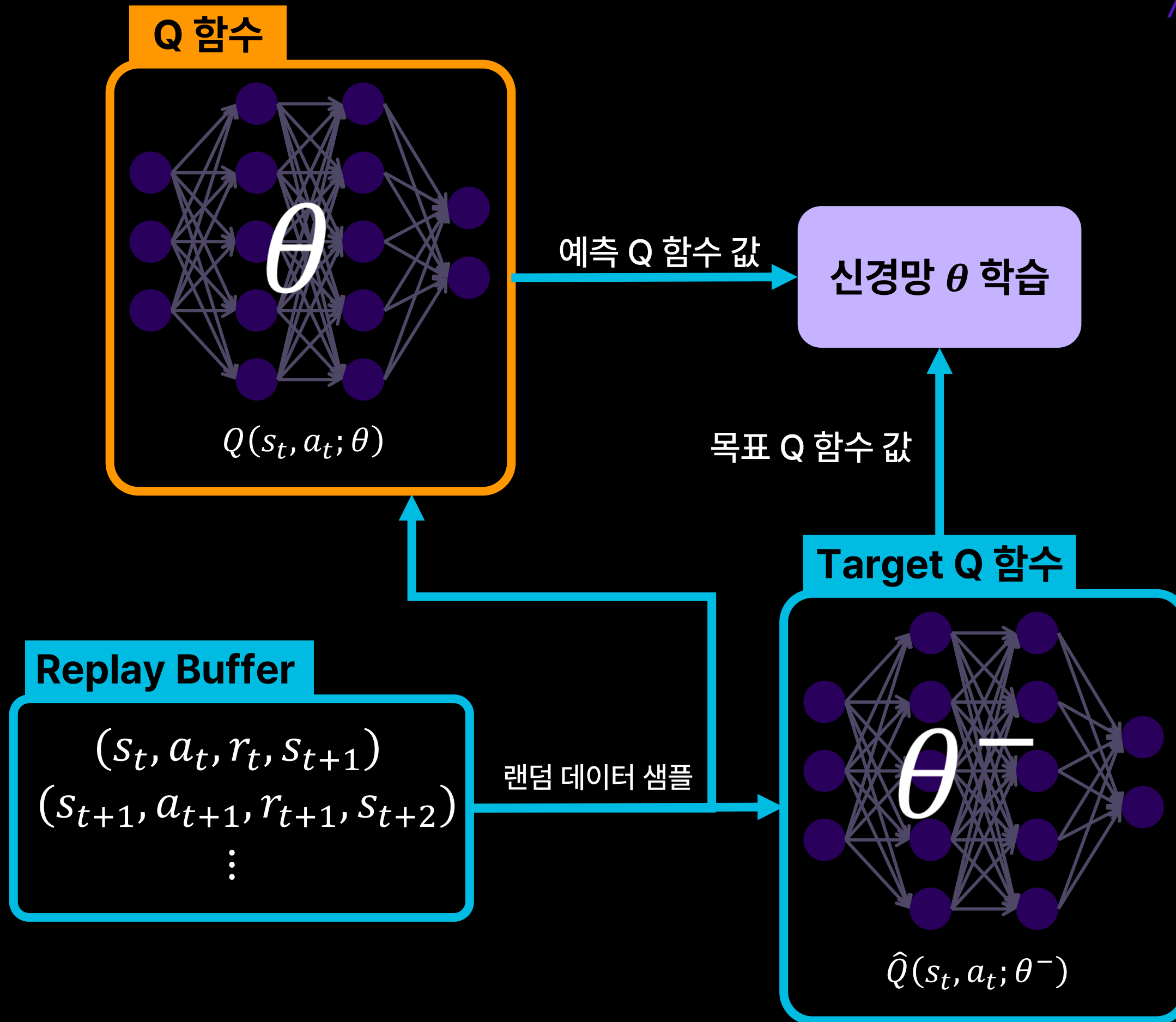
Deep Q-Network



6. Replay Buffer에서 D개의 데이터를 랜덤하게 선택하여 Q 함수와 Target Q 함수에 입력한다.



Deep Q-Network



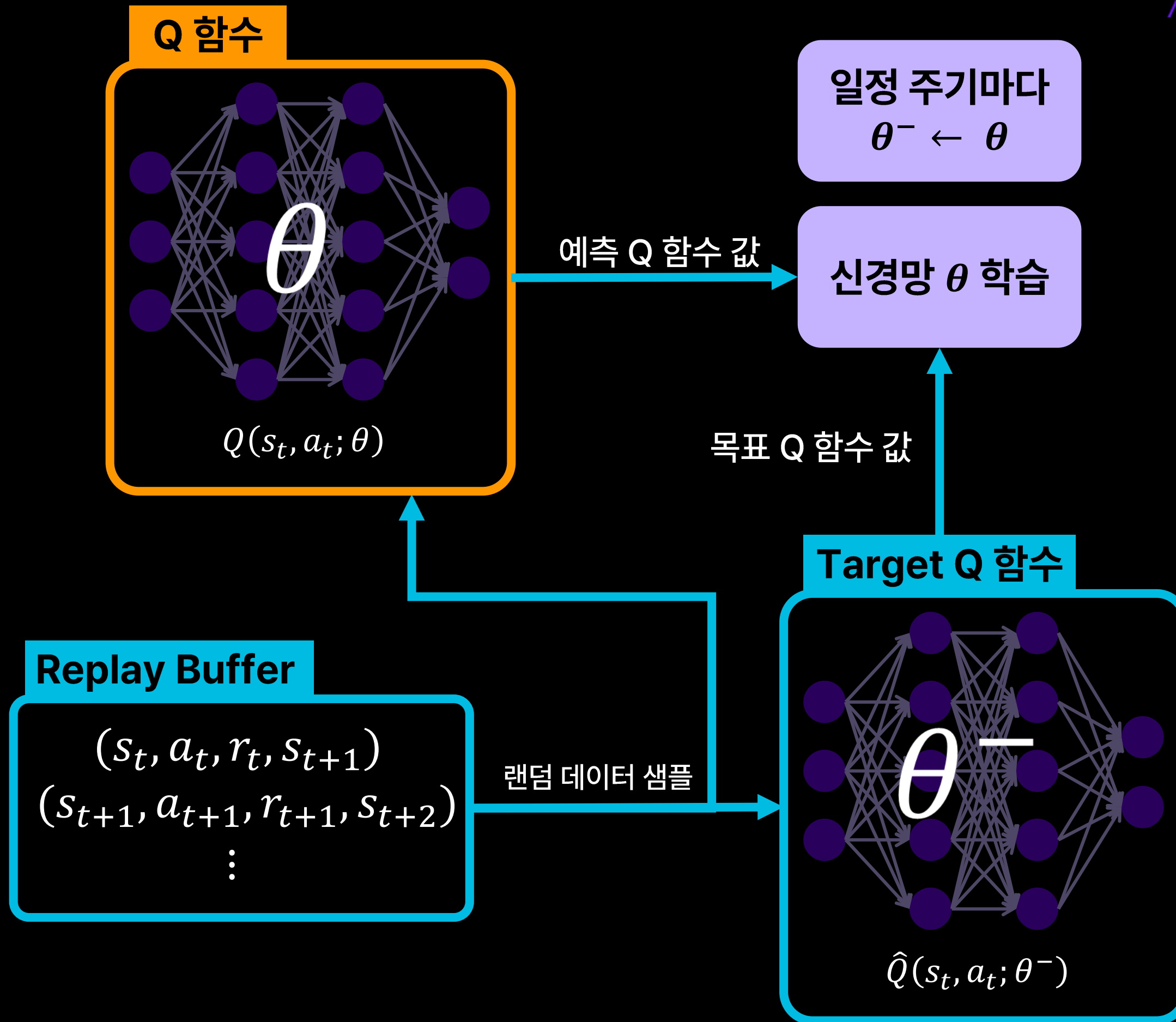
6. Replay Buffer에서 D개의 데이터를 랜덤하게 선택하여 Q 함수와 Target Q 함수에 입력한다.

7. 아래 목표함수를 기반으로 신경망 θ 를 학습시킨다.

$$\mathbb{E}[(R_t + \gamma \max_a \hat{Q}(s_{t+1}, a; \theta^-) - Q(s_t, a_t; \theta))^2]$$



Deep Q-Network



6. Replay Buffer에서 D개의 데이터를 랜덤하게 선택하여 Q 함수와 Target Q 함수에 입력한다.
7. 아래 목표함수를 기반으로 신경망 θ 를 학습시킨다.
$$\mathbb{E}[(R_t + \gamma \max_a \hat{Q}(s_{t+1}, a; \theta^-) - \hat{Q}(s_t, a_t; \theta))^2]$$
8. 1번으로 돌아간다.
9. 일정 주기마다 Target Q 함수의 신경망 θ^- 의 값을 Q 함수의 신경망 θ 의 값으로 업데이트 한다.

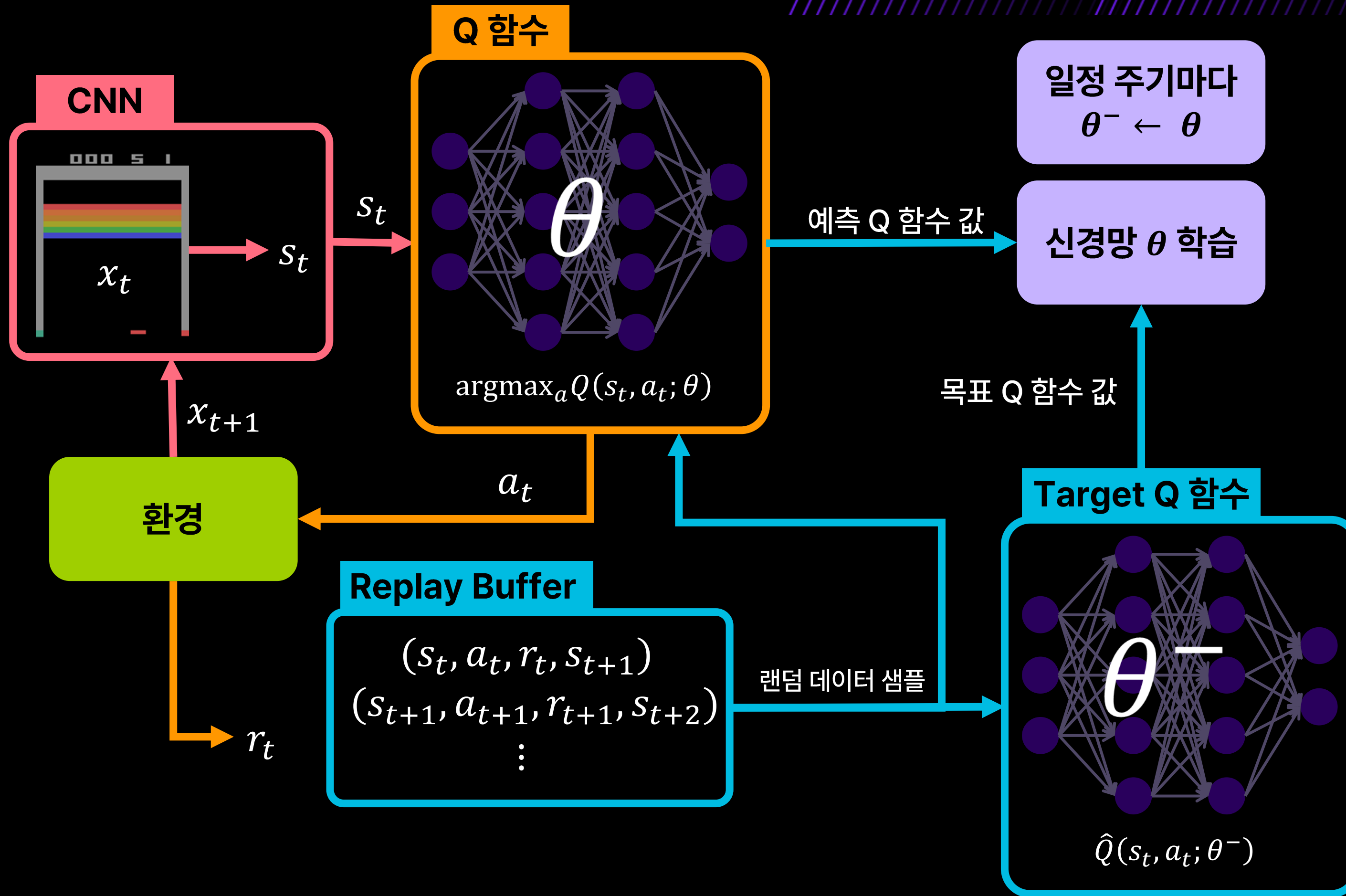


Deep Q-Network 전체 구조

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

1. Q-Learning의 한계와 DQN

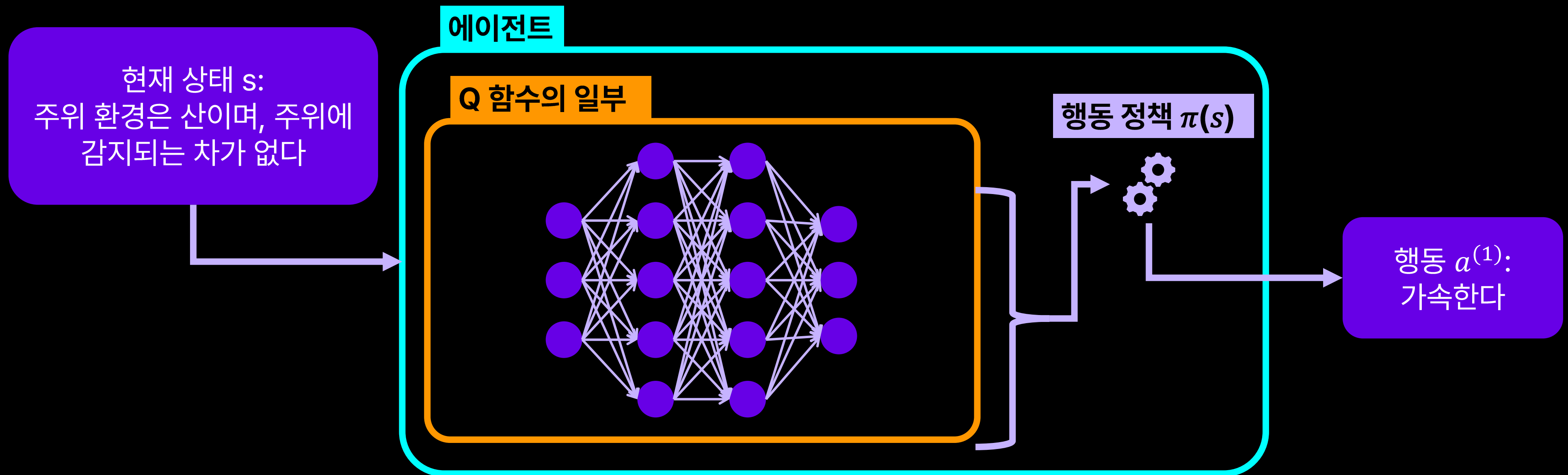


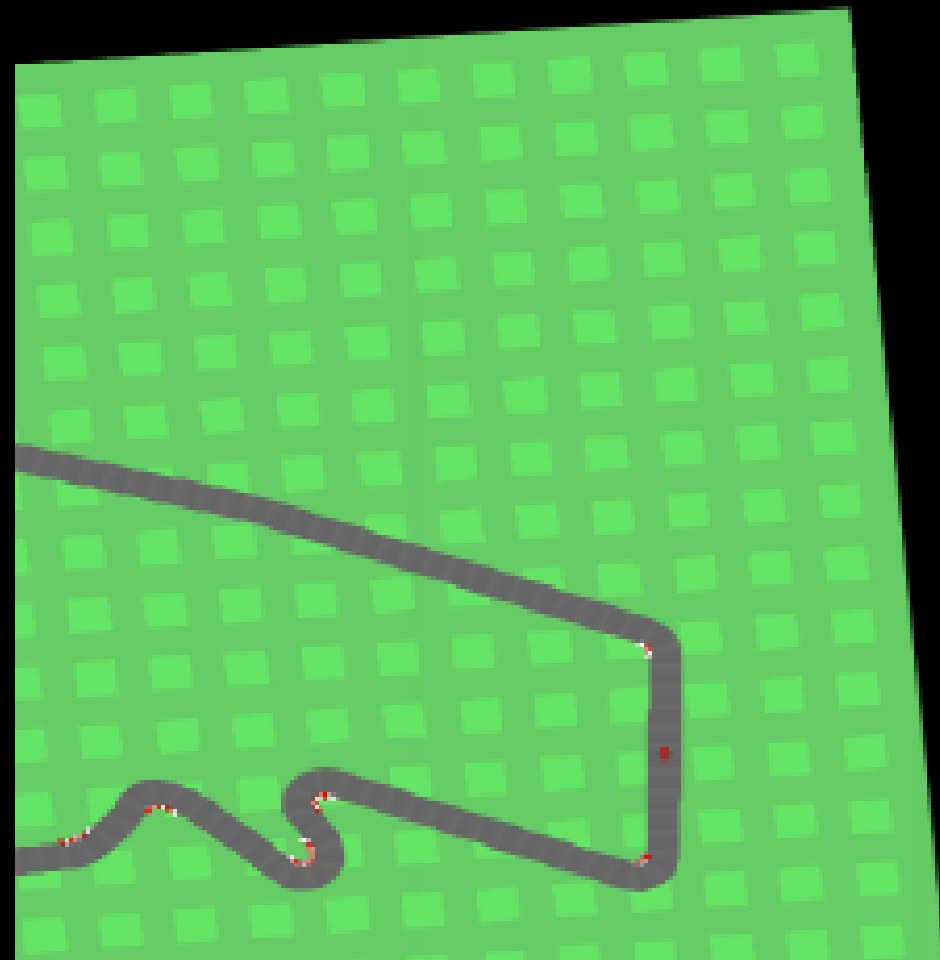
2. 정책 함수 근사



Deep Q-Network의 한계

- Deep Q-Network는 인공신경망을 통해 근사한 Q 함수를 사용한다.
- Q 함수는 각 행동의 가치를 계산하는 함수로써, **연속적인 행동 공간에서는 사용하기 어렵다.**



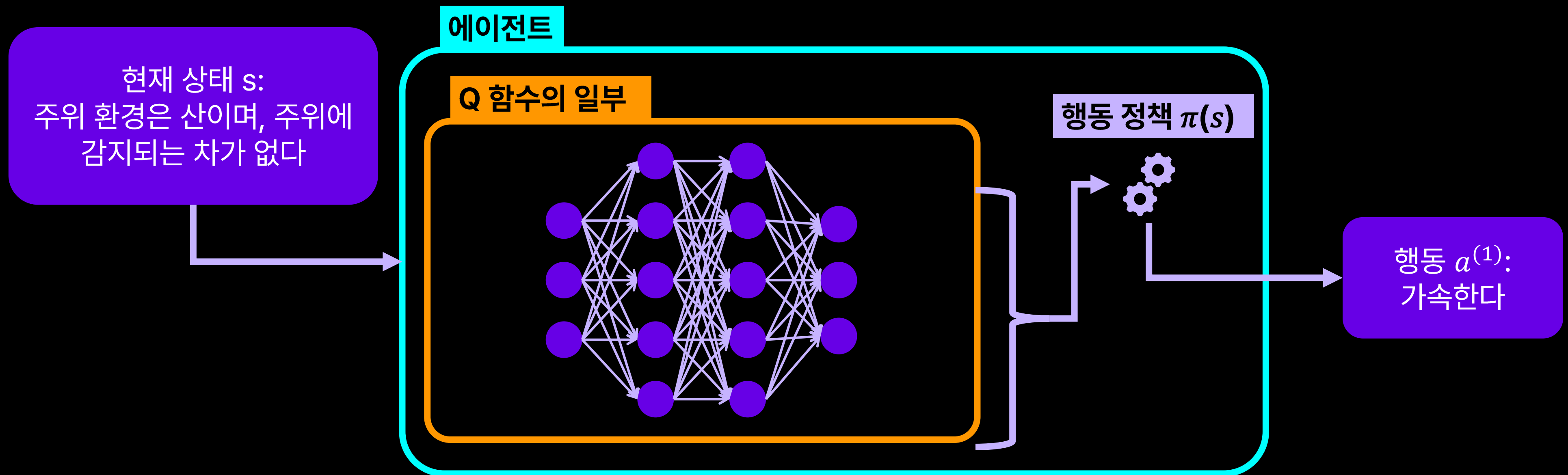


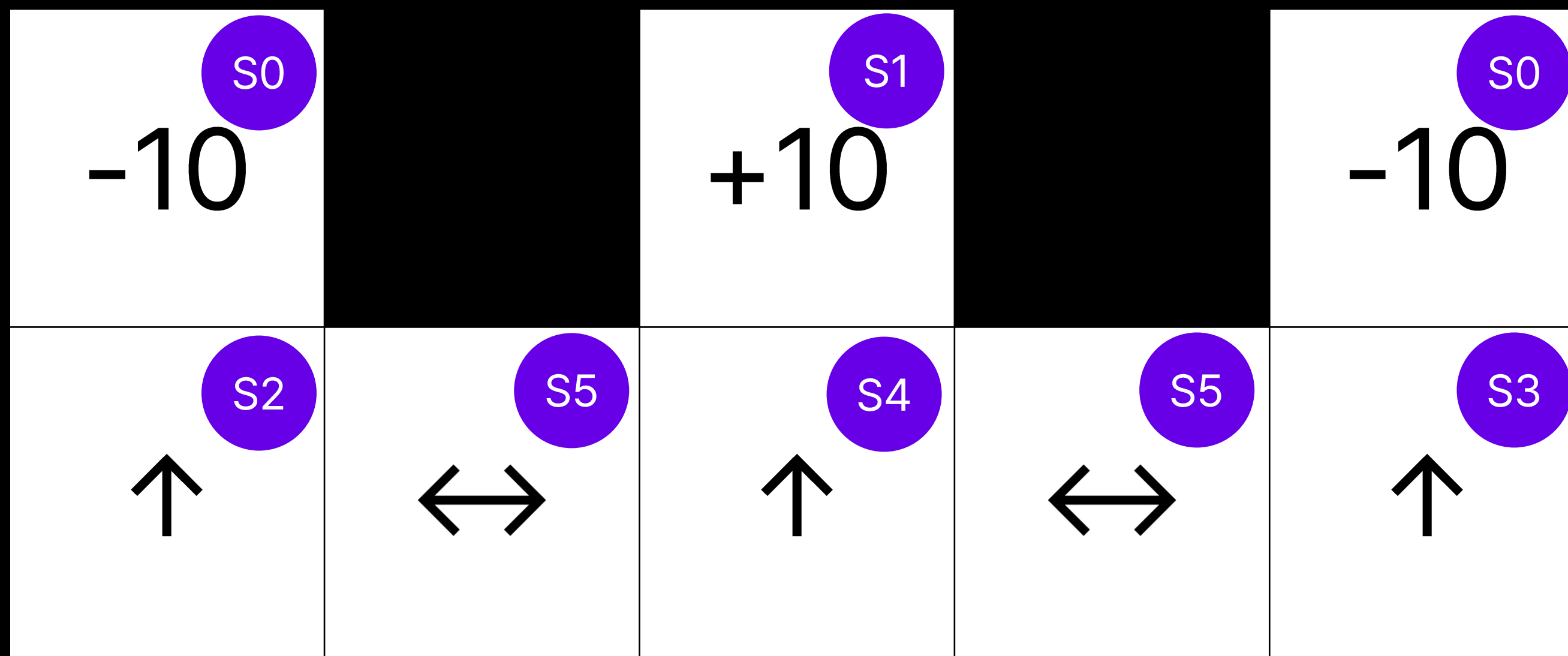
- Action: 조향 방향, 가속 세기, 감속 세기를 나타내는 **연속적인 값**
 - 조향 방향: -1 = 왼쪽으로 최대한 틀기, +1 = 오른쪽으로 최대한 틀기
 - 가속 세기: 0 = 가속하지 않음, 1 = 최대한 가속
 - 브레이크 세기: 0 = 브레이크를 밟지 않음, 1 = 최대한 브레이크를 밟음
- 한번에 여러 행동을 복합적으로 수행할 수 있다.
 - 예시: 왼쪽으로 틀면서 약하게 가속 = (-1, 0.5, 0)
- 가치를 매길 **상태 조합이 무한히 많기 때문에**, 연속적인 환경에서 Q 함수를 구현하는 것은 현실적으로 불가능하다.



Deep Q-Network의 한계

- Deep Q-Network는 인공신경망을 통해 근사한 Q 함수를 사용한다.
- Q 함수는 각 행동의 가치를 계산하는 함수로써, 연속적인 행동 공간에서는 사용하기 어렵다.
- 또한, 모든 상태에서 반드시 한 행동을 취하는 결정론적 정책이 항상 최적의 정책인 것은 아니다.





[문제 상황]

Agent는 랜덤한 위치에서 시작하며, 자신의 위치에서 상하좌우 1칸만 볼 수 있다고 가정한다.

Agent가 관측했을 때 구분할 수 없는 State는 같은 State로 간주해야 한다.

즉, 왼쪽에 있는 S5 State와 오른쪽에 있는 S5 State는 Agent 입장에서 같은 State이다.



Deep Q-Network의 한계

LiDAR 센서와 강화학습

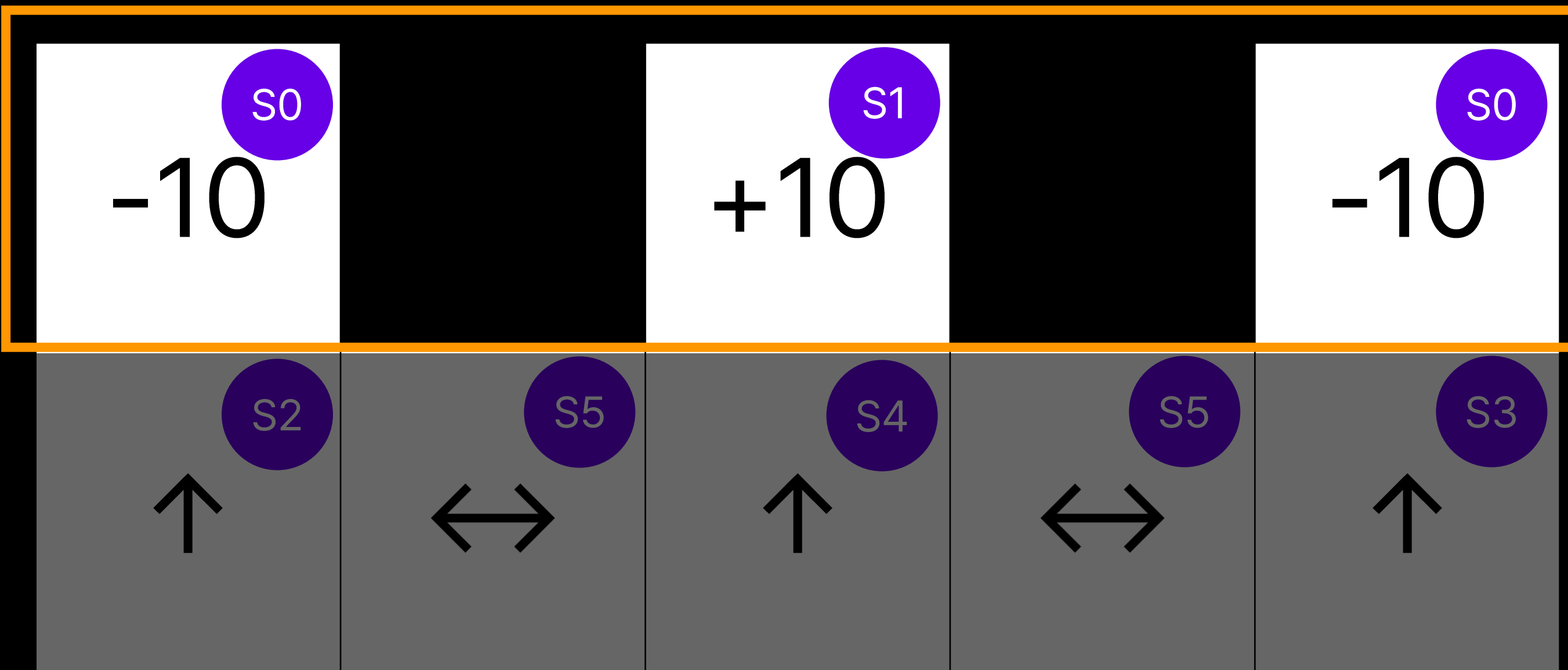
03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사

S0, S1 State는 Terminal State이다.

S2, S3 State에서는 각각 오른쪽, 왼쪽으로 가는 것이 최적의 행동 정책이다.

S4 State에서는 위로 이동하는 것이 최적의 행동 정책이다.



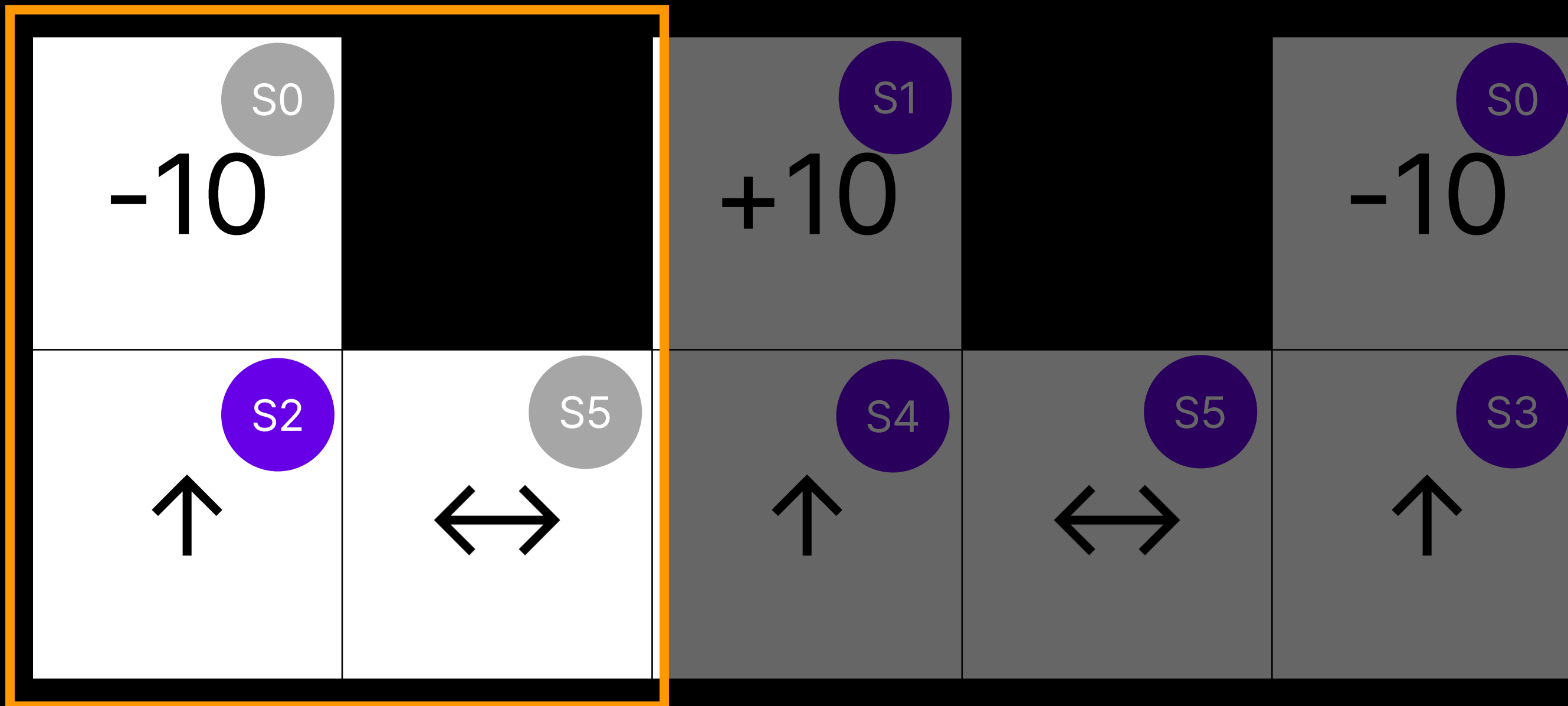


Deep Q-Network의 한계

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사



S0, S1 State는 Terminal State이다.

S2, S3 State에서는 각각 오른쪽, 왼쪽으로 가는 것이 최적의 행동 정책이다.

S4 State에서는 위로 이동하는 것이 최적의 행동 정책이다.

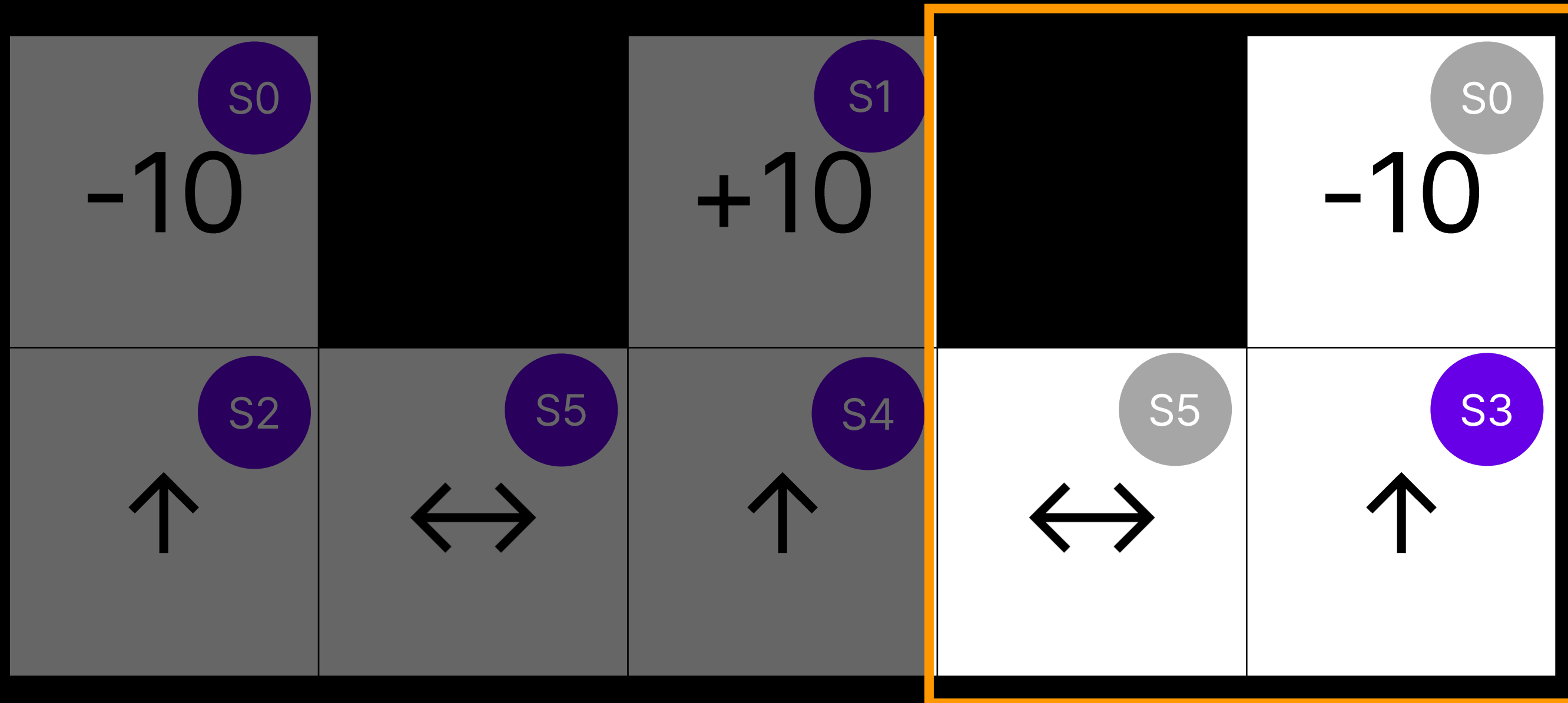


Deep Q-Network의 한계

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사



S0, S1 State는 Terminal State이다.

S2, S3 State에서는 각각 오른쪽, 왼쪽으로 가는 것이 최적의 행동 정책이다.

S4 State에서는 위로 이동하는 것이 최적의 행동 정책이다.

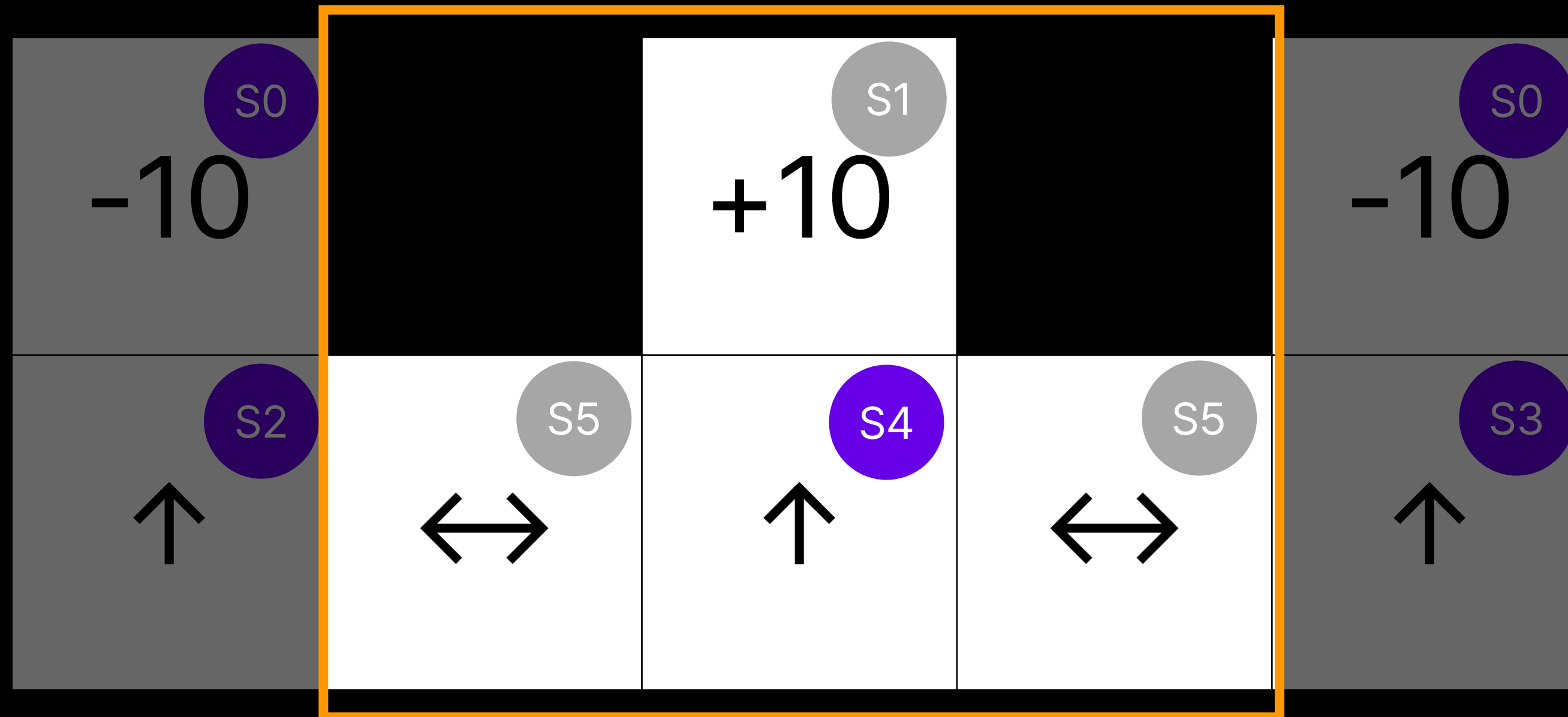


Deep Q-Network의 한계

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사



S0, S1 State는 Terminal State이다.

S2, S3 State에서는 각각 오른쪽, 왼쪽으로 가는 것이 최적의 행동 정책이다.

S4 State에서는 위로 이동하는 것이 최적의 행동 정책이다.

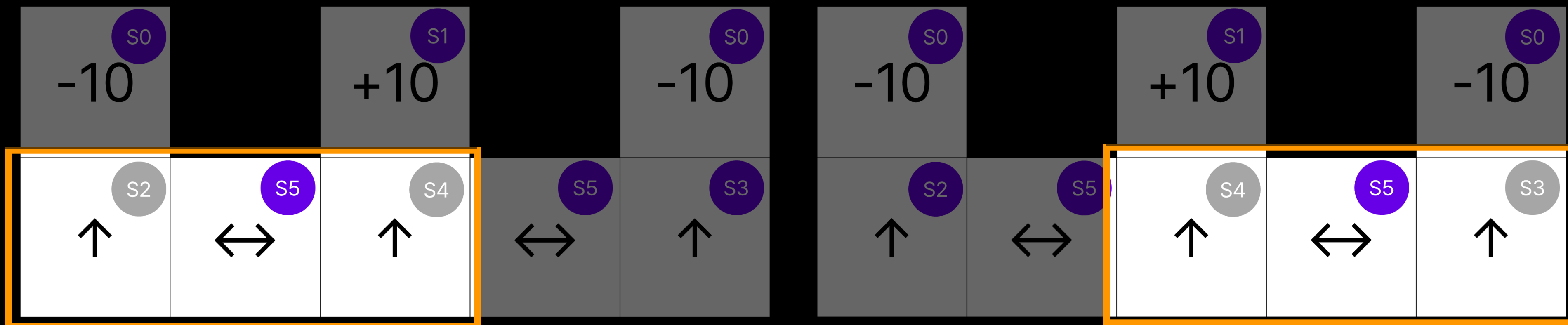


Deep Q-Network의 한계

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사



S5 State

Agent는 두 경우의 수를 구분할 수 없다. (Agent는 State 번호를 볼 수 없고, 화살표나 보상만 볼 수 있다)

$Q(S5, \leftarrow)$ 와 $Q(S5, \rightarrow)$ 중 어느 쪽의 값이 더 높건 간에, 보상을 얻을 수 없는 경우가 생긴다.

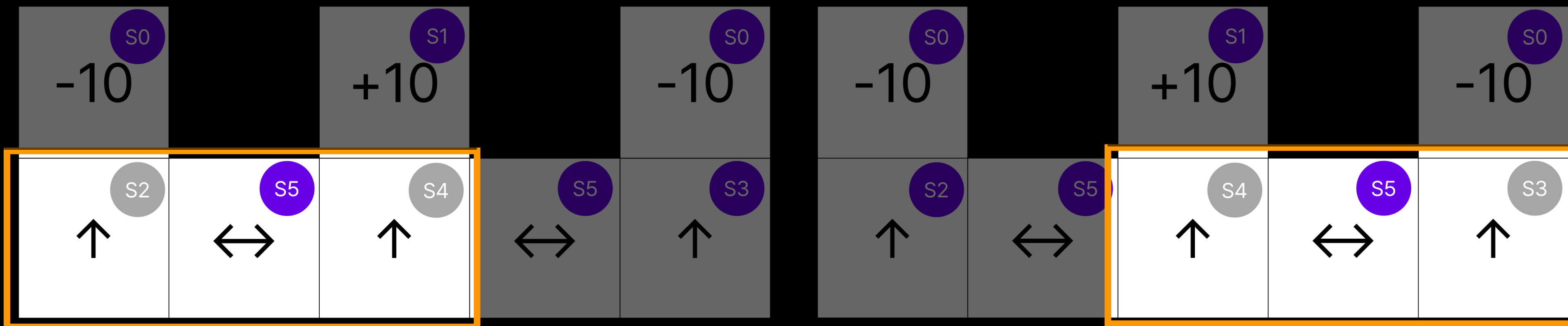


Deep Q-Network의 한계

LiDAR 센서와 강화학습

03 다양한 강화학습 알고리즘 적용

2. 정책 함수 근사

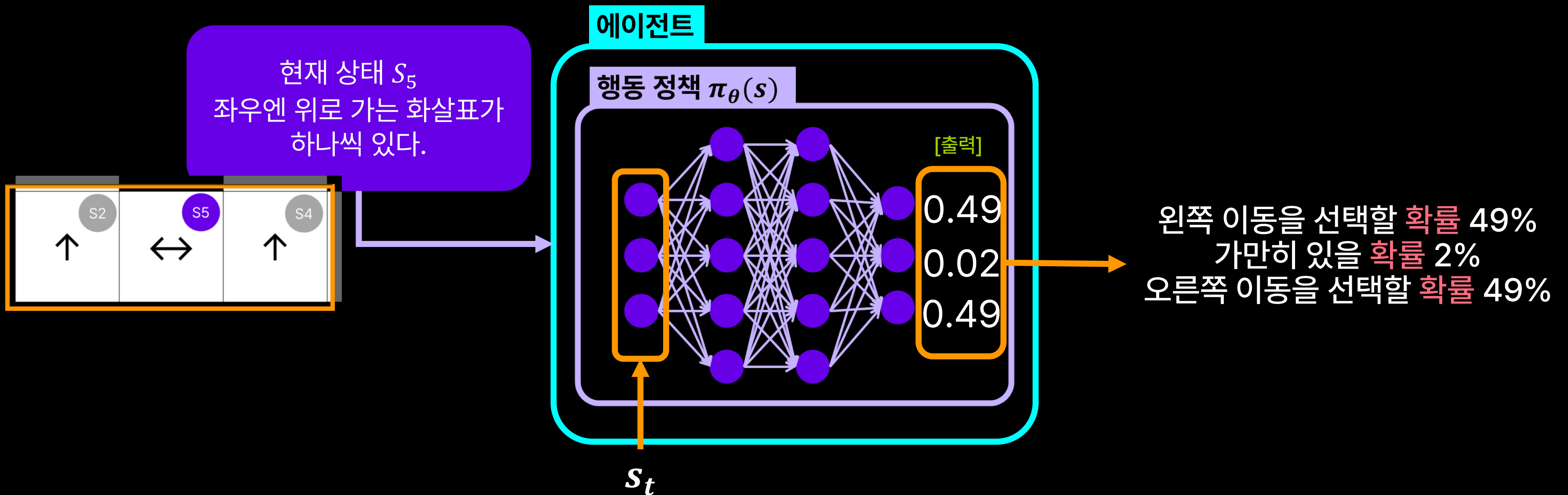


50%의 확률로 왼쪽, 나머지는 오른쪽으로 이동한다면 언젠가는 S1 에 도착할 수 있다.

그러나 Q함수를 활용하는 정책 함수는 위와 같은 정책을 도출할 수 없다.

Q 함수를 사용하지 않고 확률론적 결정이 가능한 정책 함수로 대체하면 어떨까?

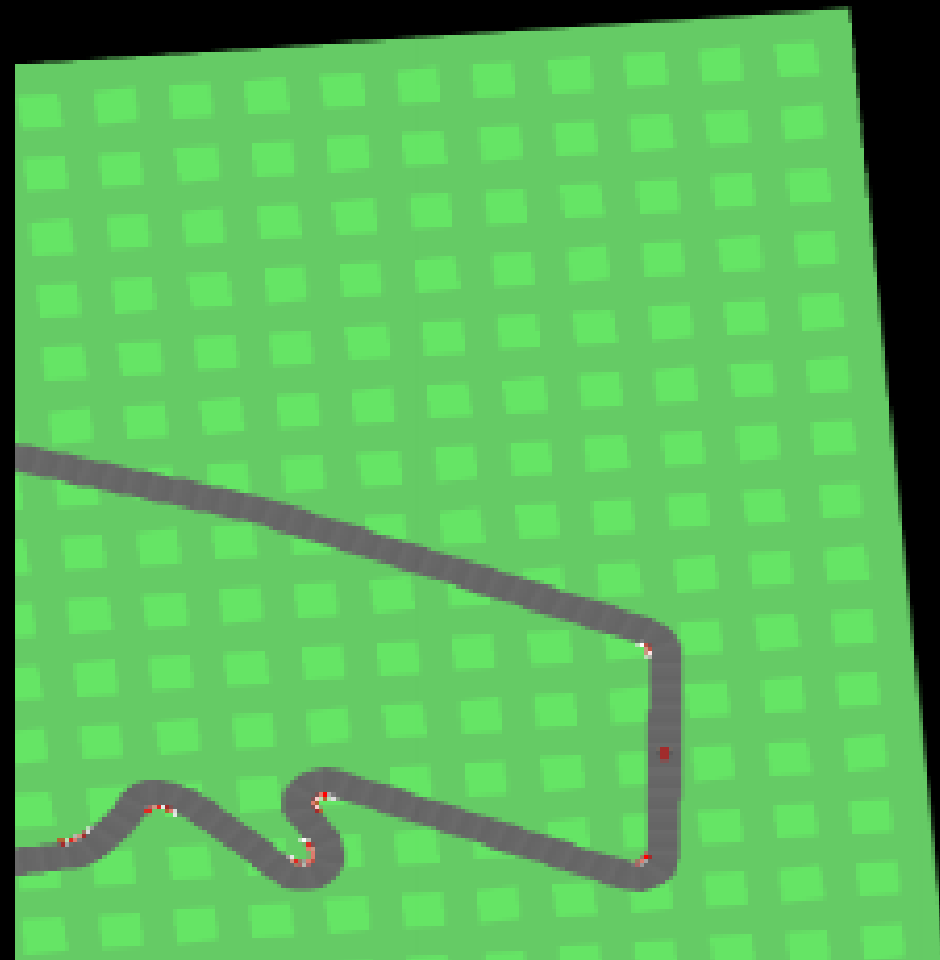
Q 함수를 사용하지 않고 **확률론적 결정이 가능한 정책 함수**로 대체하면 어떨까?



엄밀하게는 파라미터화 된(parameterized) 행동 정책 $\pi_{\theta}(s)$ 이라고 칭해야 하나, 이해를 돕기 위해 인공신경망 기반 행동 정책이라고 칭하겠습니다.

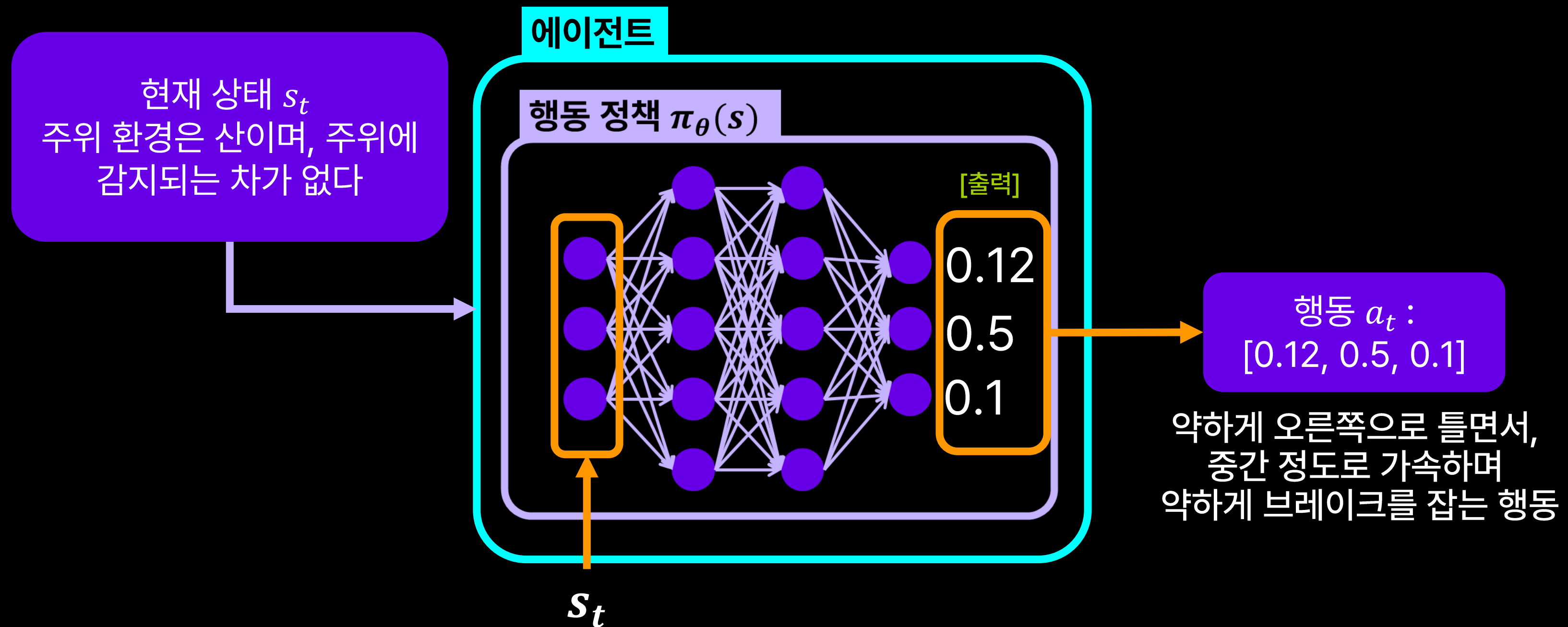


- Action: 조향 방향, 가속 세기, 감속 세기를 나타내는 **연속적인 값**
 - 조향 방향: -1 = 왼쪽으로 최대한 틀기, +1 = 오른쪽으로 최대한 틀기
 - 가속 세기: 0 = 가속하지 않음, 1 = 최대한 가속
 - 브레이크 세기: 0 = 브레이크를 밟지 않음, 1 = 최대한 브레이크를 밟음



- 한번에 여러 행동을 복합적으로 수행할 수 있다.
 - 예시: 왼쪽으로 틀면서 약하게 가속 = (-1, 0.5, 0)
- 가치를 매길 **상태 조합이 무한히 많기 때문에**, 연속적인 환경에서 Q 함수를 구현하는 것은 현실적으로 불가능하다.

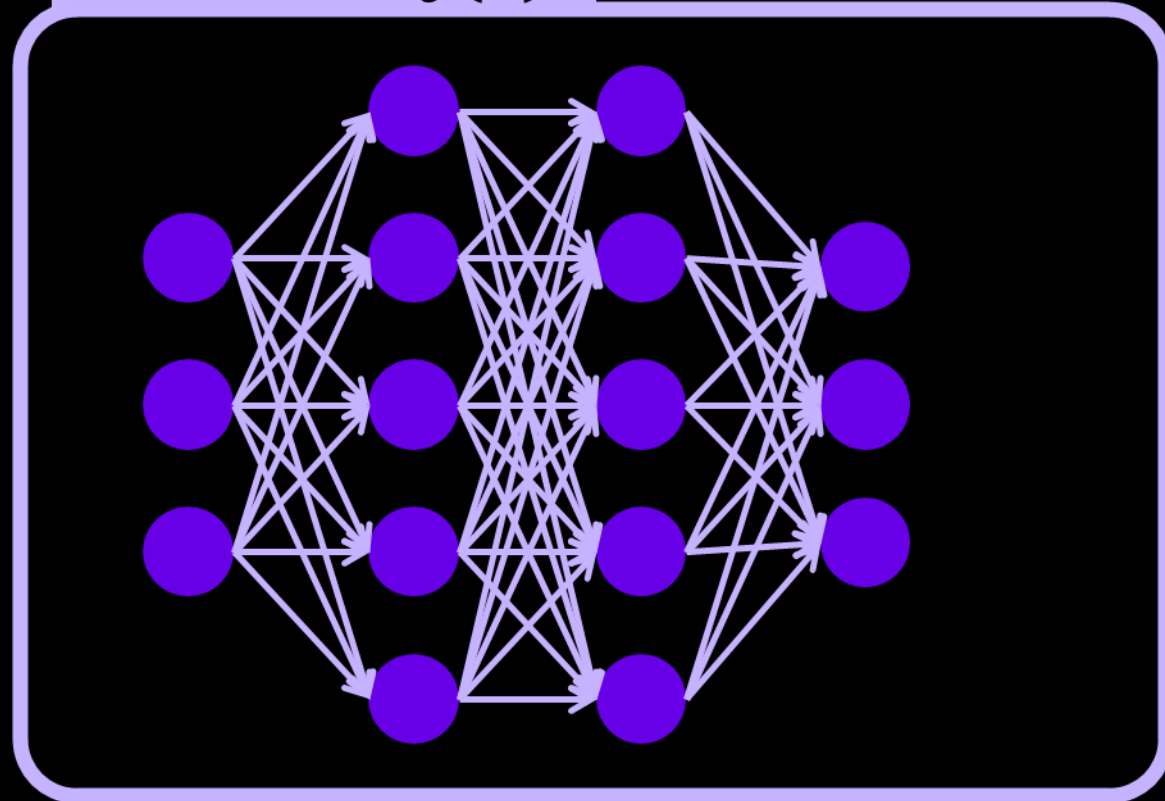
- Action: 조향 방향, 가속 세기, 감속 세기를 나타내는 **연속적인 값**
 - 조향 방향: -1 = 왼쪽으로 최대한 틀기, +1 = 오른쪽으로 최대한 틀기
 - 가속 세기: 0 = 가속하지 않음, 1 = 최대한 가속
 - 브레이크 세기: 0 = 브레이크를 밟지 않음, 1 = 최대한 브레이크를 밟음



엄밀하게는 파라미터 기반 행동 정책 $\pi_{\theta}(s)$ 이라고 칭해야 하나, 이해를 돕기 위해 인공신경망 기반 행동 정책이라고 칭합니다.

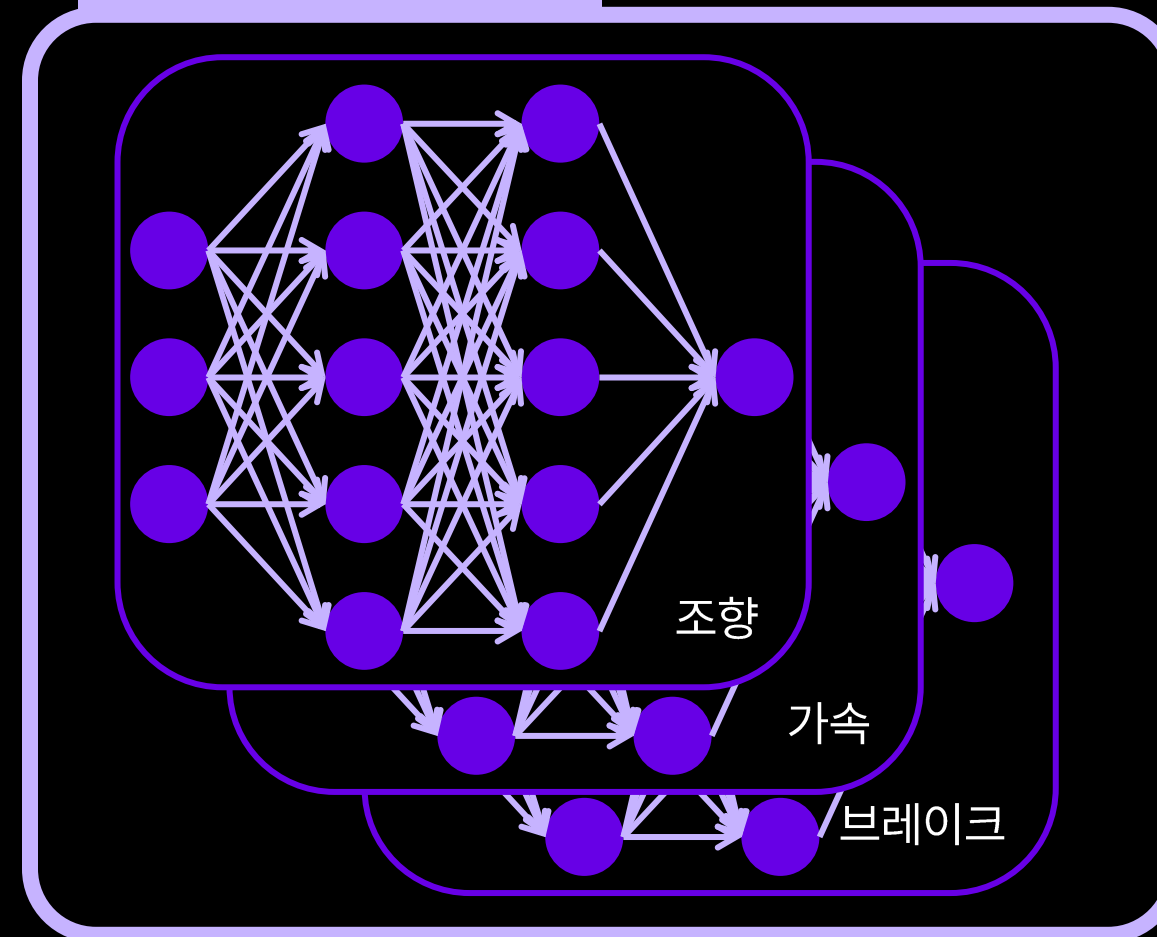


행동 정책 $\pi_{\theta}(s)$



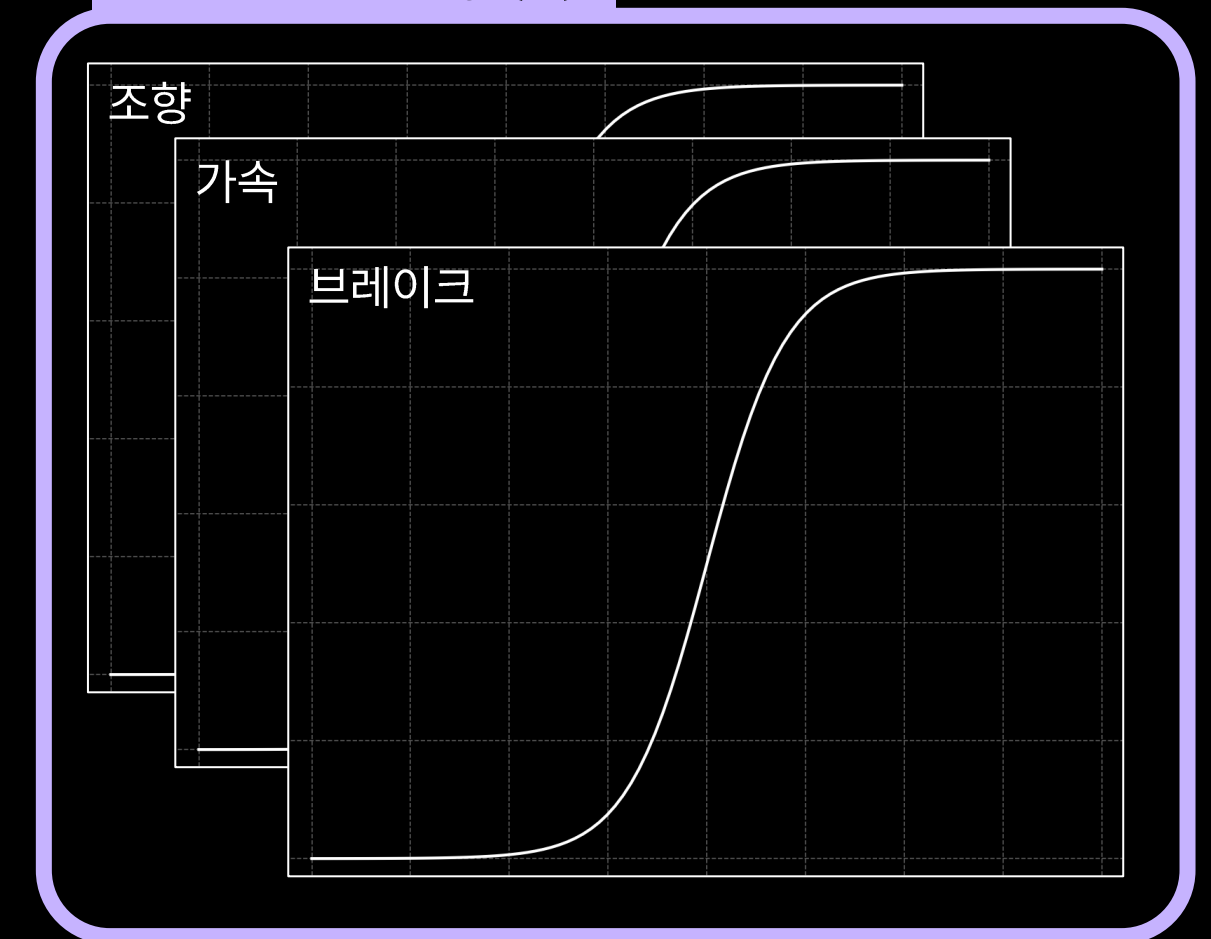
단일 인공신경망 행동 정책

행동 정책 $\pi_{\theta}(s)$



다중, 다층 인공신경망 행동 정책

행동 정책 $\pi_{\theta}(s)$



(머신러닝) 로지스틱 회귀를 활용한
행동 정책



- Deep Q Network에서 사용한 목표 함수

$$J_t(\theta) = \mathbb{E}[(R_t + \gamma \max_a \hat{Q}(s_{t+1}, a; \theta^-) - Q(s_t, a_t; \theta))^2]$$

↓
목표
Q 함수 값

↓
예측
Q 함수 값

- 정책 함수 근사 기법에서는 Q 함수를 사용하지 않으므로, 다른 목표 함수를 사용해야 한다.

- Q 함수를 단순히 정책 함수로 대체할 경우, 의미가 없는 식이 된다.

$$J_t(\theta) = \mathbb{E}[(R_t + \gamma \max_a \hat{\pi}(s_{t+1}; \theta^-) - \pi(s_t; \theta))^2]$$

↓
상태 s_{t+1} 에서 정책 $\hat{\pi}$ 가
선택한 행동 중 가장 큰
행동과 보상의 합

↓
상태 s_t 에서
정책 $\hat{\pi}$ 가
선택한 행동



목표: 정책을 따랐을 때 **보상의 기댓값이 최대**가 되도록 파라미터를 조정한다.

→ 전체 **결과가 좋다면 같은 행동을 할 확률을 높이고**, 그렇지 않다면 낮춘다.

초기 상태 s_0 에서 정책 $\hat{\pi}$ 를 따라 k 번 행동한 궤적을 $\tau_k = (s_0, \hat{\pi}(s_0), s_1, \dots, s_{k+1})$ 이라 할 때, **목적 함수** $J(\theta)$ 를 다음과 같이 정의할 수 있다.

$$J(\theta) = \mathbb{E}[R(\tau_k)]$$



궤적 τ_k 를 따랐을 때 얻은 보상의 총합의 기댓값

=

초기 상태 s_0 에서 정책 $\hat{\pi}$ 를 따랐을 때 얻은
보상의 총합의 기댓값



초기 상태 s_0 에서 정책 $\hat{\pi}$ 를 따랐을 때 **얻은 보상의 총합의 기댓값이 최대가 되도록** 정책 함수를 학습시킨다.

딥러닝에서 사용하는 경사하강법은 최적화 하고자 하는 목적 함수의 **Gradient (그라디언트, 경사)**를 따라 파라미터를 조정하며 최적화하는 기법이다.

이를 그대로 적용하기 위해, 목적 함수의 그라디언트를 계산해야 한다.

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}[R(\tau_k)]$$



Policy Gradient Theorem에 따라 목적 함수의 Gradient (경사) 를 아래와 같이 나타낼 수 있다.

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(s, a) R(\tau_k)]$$

궤적의 각 상태 별 정책 함수가
선택한 행동에 대한 로그 확률

궤적 τ_k 를
따랐을 때 얻은
누적 보상



Policy Gradient Theorem의 증명 과정은
[보조자료 1] 을 확인해주세요



Policy Gradient Theorem에 따라 목적 함수의 Gradient (경사) 를 아래와 같이 나타낼 수 있다.

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(s, a) R(\tau_k)]$$

각 상태에서 정책 함수가 내리는
판단의 방향성

정책 함수의
모든 판단에
대한 보상의 합

→ 전체 궤적의 결과가 좋다면 궤적 τ_k 과 같은 행동을 할 확률을 높이고, 그렇지 않다면 낮춘다.

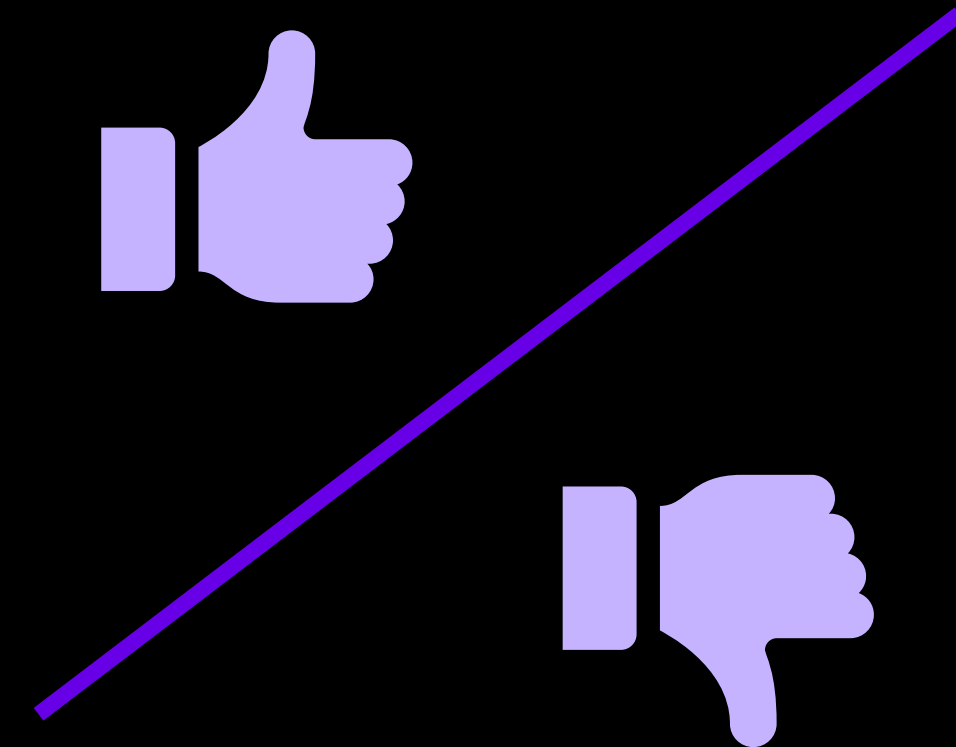
3. Actor–Critic



- Policy Gradient 알고리즘은 전체 궤적의 **결과가 좋다면 그 궤적을 결정한 파라미터를 강화하고, 그렇지 않다면 약화**한다.
- 전체 궤적을 한번에 평가하기 때문에 아래 문제가 발생한다.
 - 정확히 **어느 행동**이 큰 영향을 미쳤는지 알기 어렵다.
 - 보상의 분산(Variance)이 크다면 학습 효과가 떨어질 수 있다.
- 전체 궤적을 한번에 평가하지 말고, 각 행동을 평가하여 최적화 할 수는 없을까?



전체 과정에 대해 한번에 평가하는 대신, 각 행동의 가치를 평가한다.



Actor – 정책 함수를 바탕으로 **행동을 선택**

Critic – Q 함수 (가치 함수)를 바탕으로 **행동을 평가**



Actor – Critic 학습 과정

에이전트(Actor)

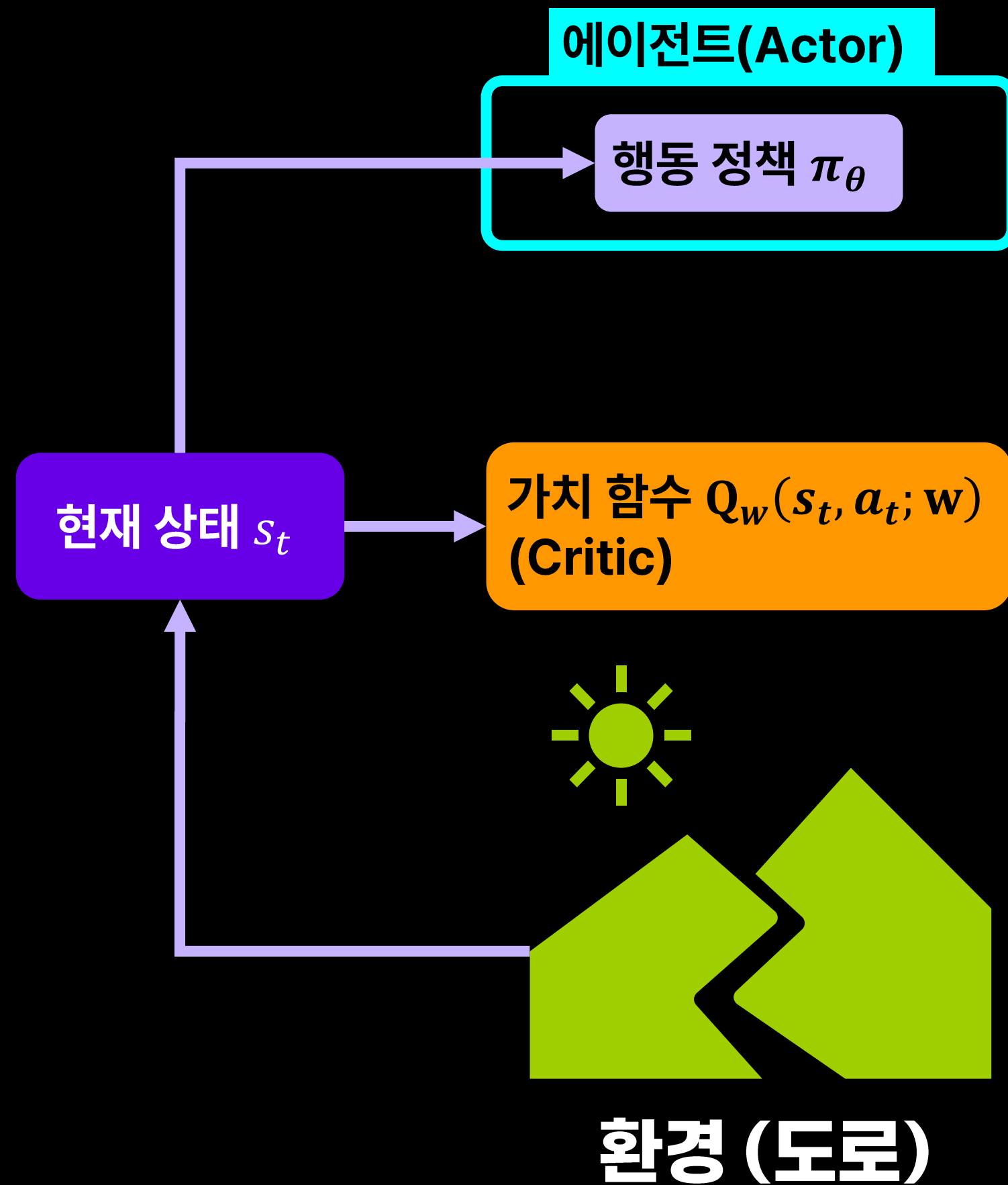
행동 정책 π_{θ}

가치 함수 $Q_w(s_t, a_t; w)$
(Critic)

1. Actor와 Critic의 파라미터를 랜덤하게 초기화 한다.
 1. Actor의 파라미터를 θ 로 나타낸다
 2. Critic의 파라미터를 w 로 나타낸다
2. 현재 상태를 Actor와 Critic에 입력한다.
3. Actor의 Policy가 행동을 출력한다.
4. 출력한 행동을 가치 함수에 입력하고, 환경에서 수행한다.



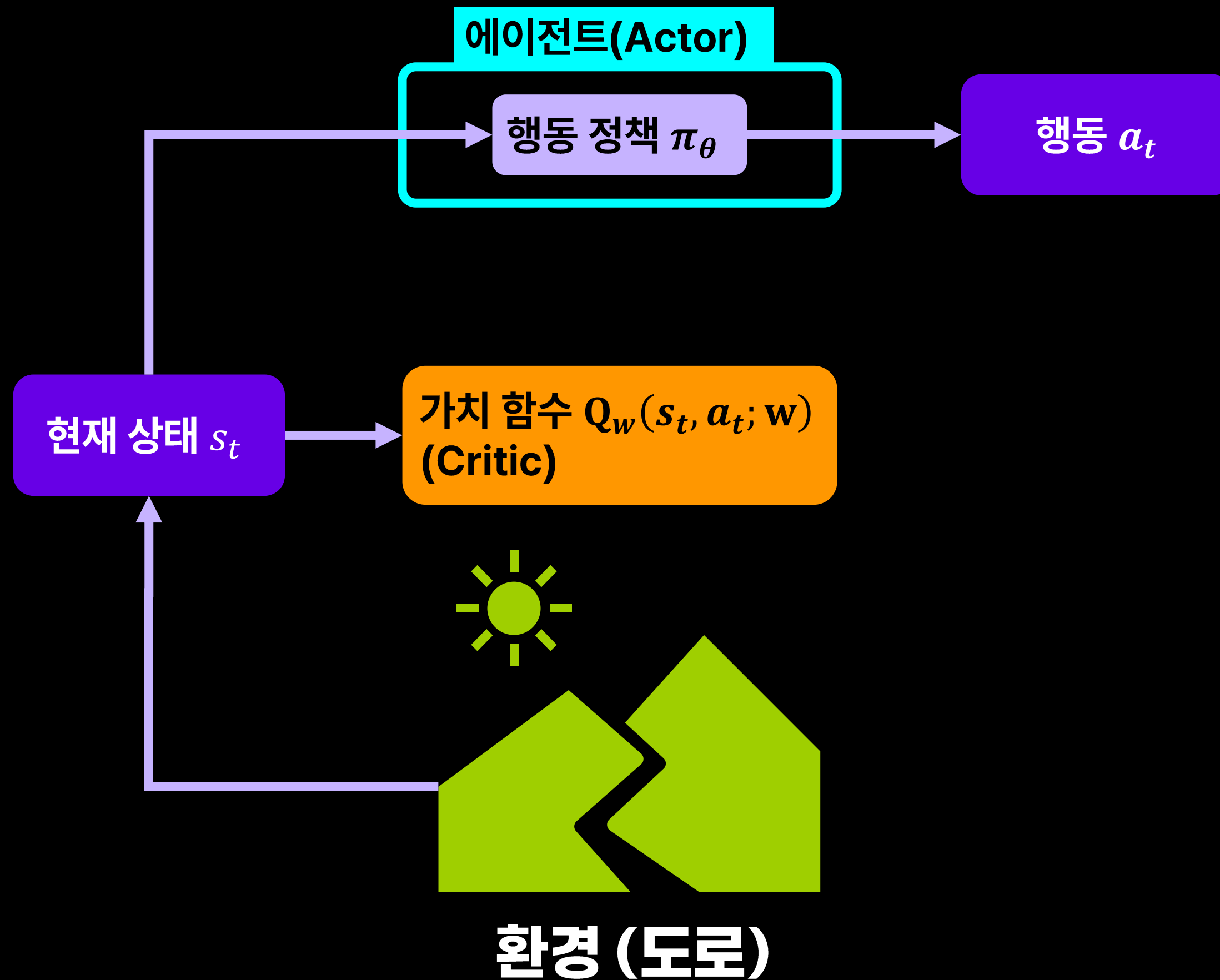
Actor – Critic 학습 과정



1. Actor와 Critic의 파라미터를 랜덤하게 초기화 한다.
 1. Actor의 파라미터를 θ 로 나타낸다
 2. Critic의 파라미터를 w 로 나타낸다
2. 현재 상태를 Actor와 Critic에 입력한다.
3. Actor의 Policy가 행동을 출력한다.
4. 출력한 행동을 가치 함수에 입력하고, 환경에서 수행한다.



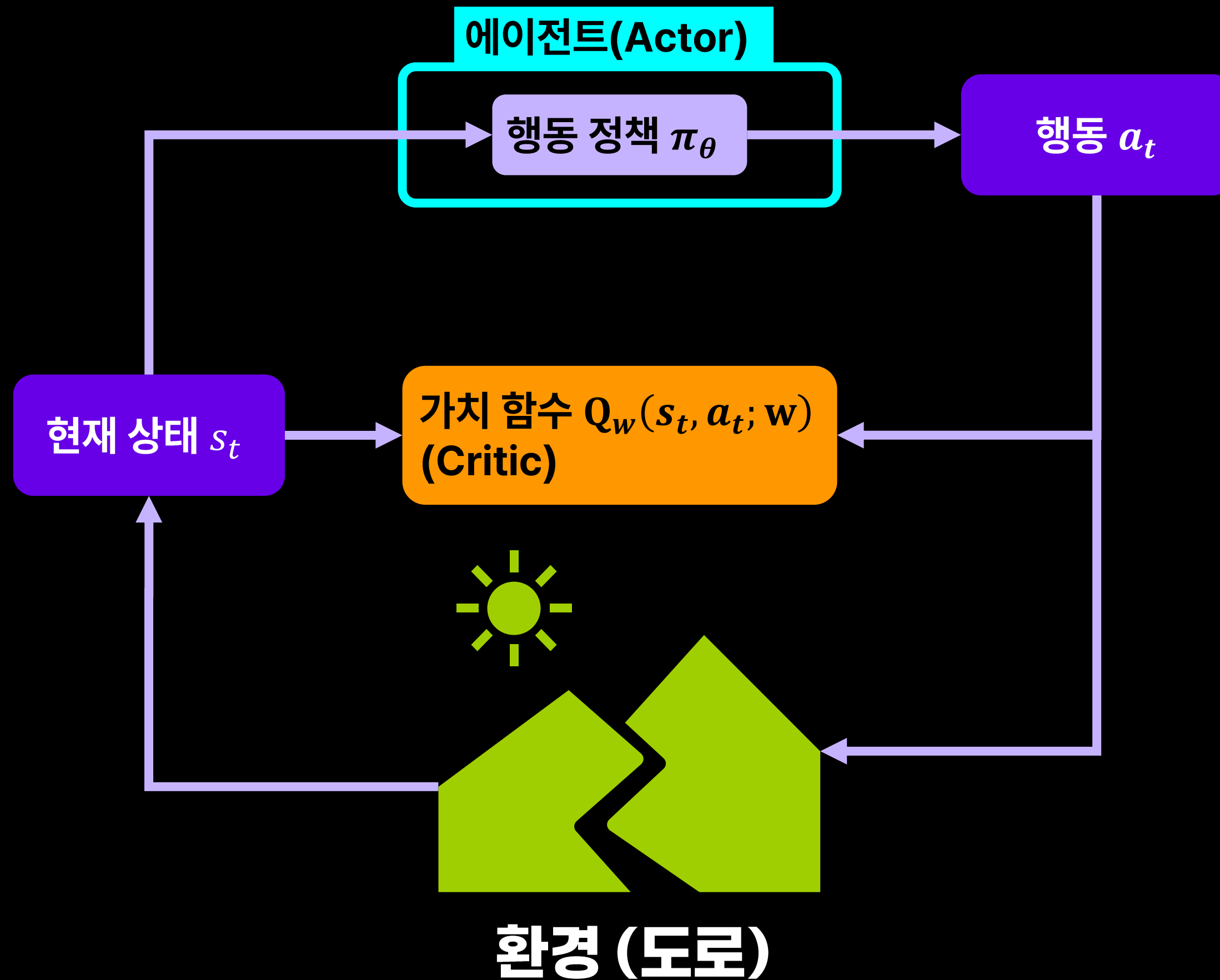
Actor – Critic 학습 과정



1. Actor와 Critic의 파라미터를 랜덤하게 초기화 한다.
 1. Actor의 파라미터를 θ 로 나타낸다
 2. Critic의 파라미터를 w 로 나타낸다
2. 현재 상태를 Actor와 Critic에 입력한다.
3. Actor의 Policy가 행동을 출력한다.
4. 출력한 행동을 가치 함수에 입력하고, 환경에서 수행한다.



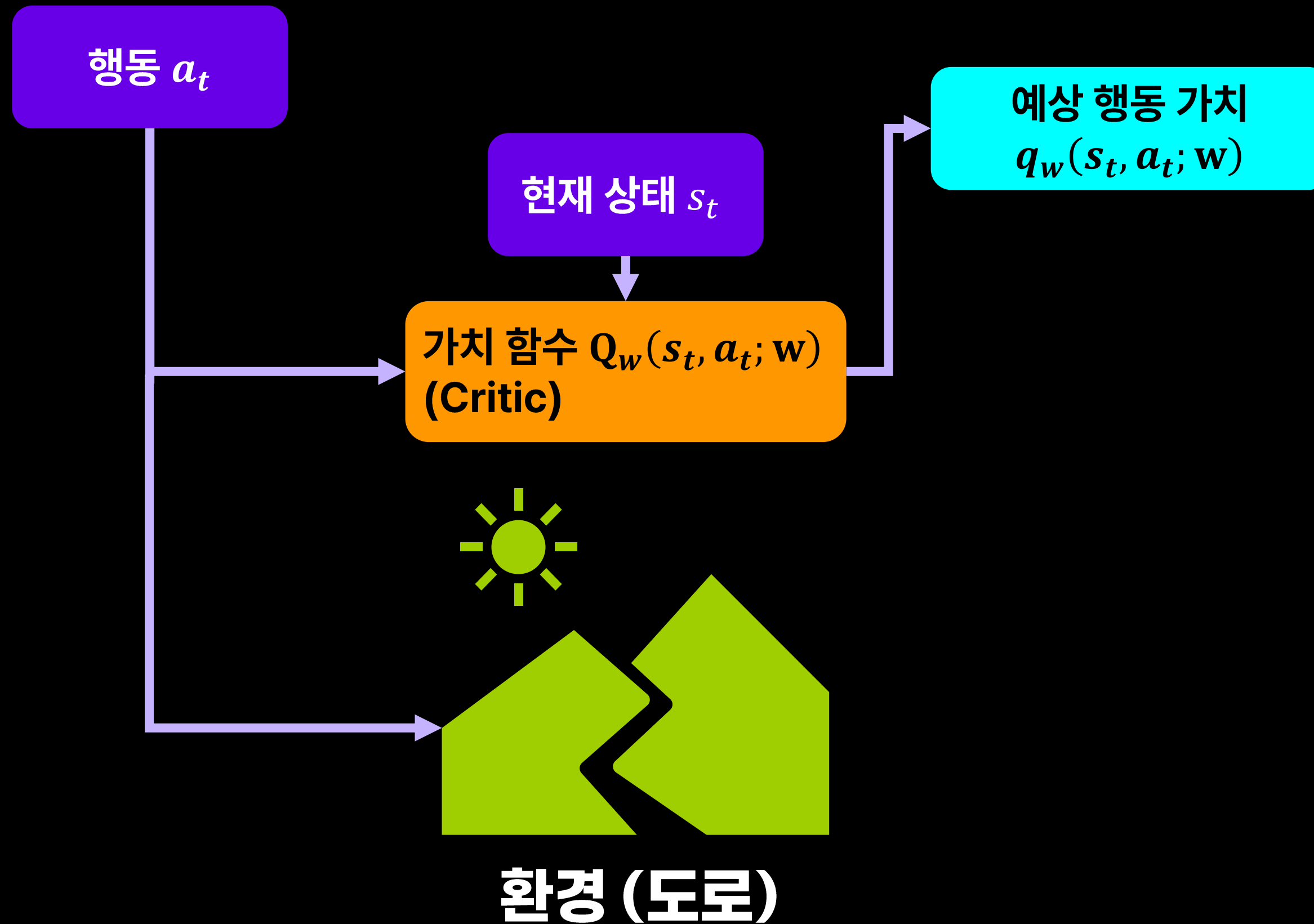
Actor – Critic 학습 과정



1. Actor와 Critic의 파라미터를 랜덤하게 초기화 한다.
 1. Actor의 파라미터를 θ 로 나타낸다
 2. Critic의 파라미터를 w 로 나타낸다
2. 현재 상태를 Actor와 Critic에 입력한다.
3. Actor의 Policy가 행동을 출력한다.
4. 출력한 행동을 가치 함수에 입력하고, 환경에서 수행한다.



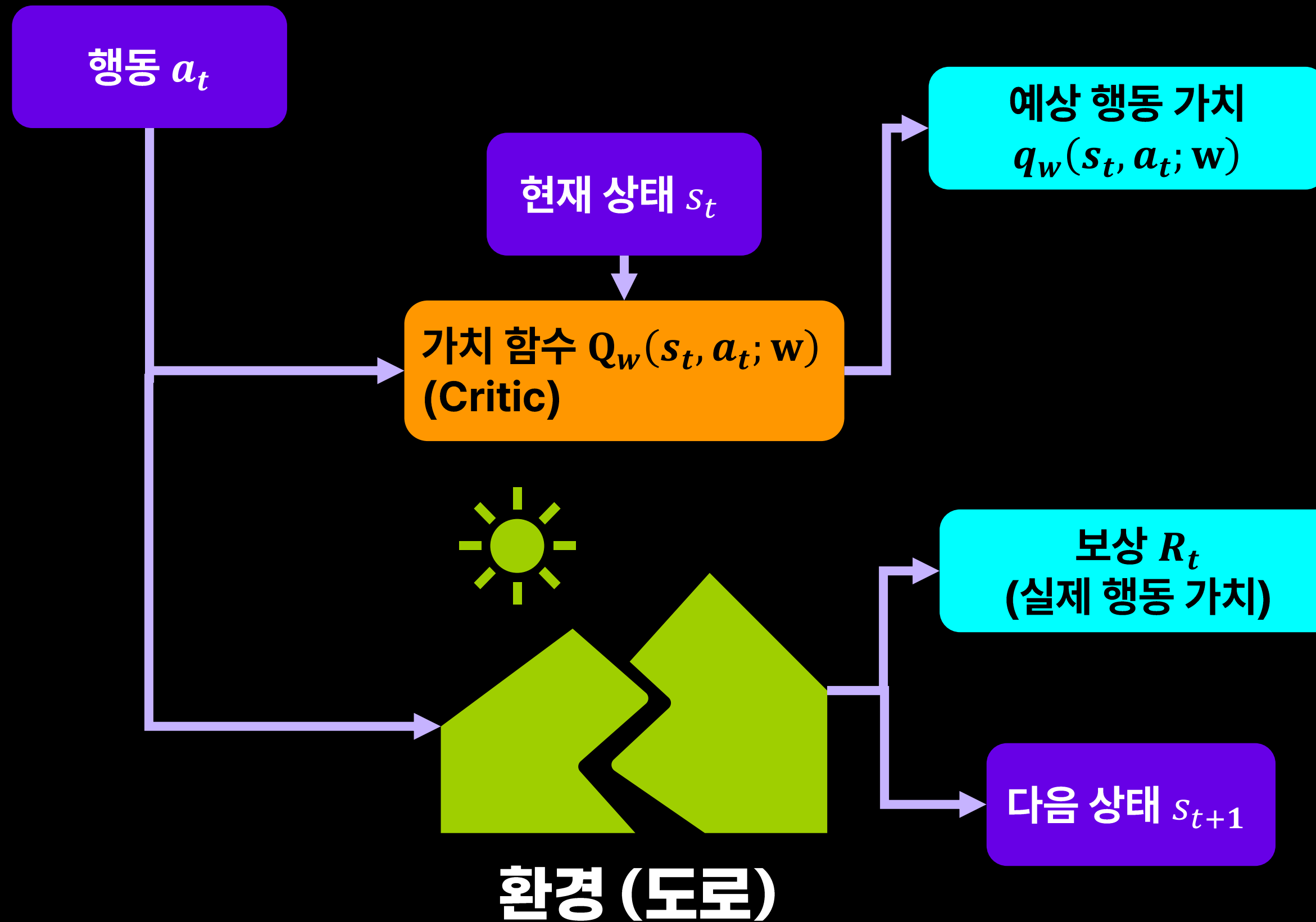
Actor – Critic 학습 과정



5. 가치 함수는 현재 상태와 행동을 입력 받아 Q 함수 값 (예상 행동 가치)을 출력한다.
6. 환경은 행동에 따라 보상을 제공하고, 다음 상태로 넘어간다.
7. 예상 행동 가치를 바탕으로 정책 함수를 업데이트 한다.
8. 예상 행동 가치와 보상 값의 차이를 기반으로 가치 함수를 업데이트 한다.



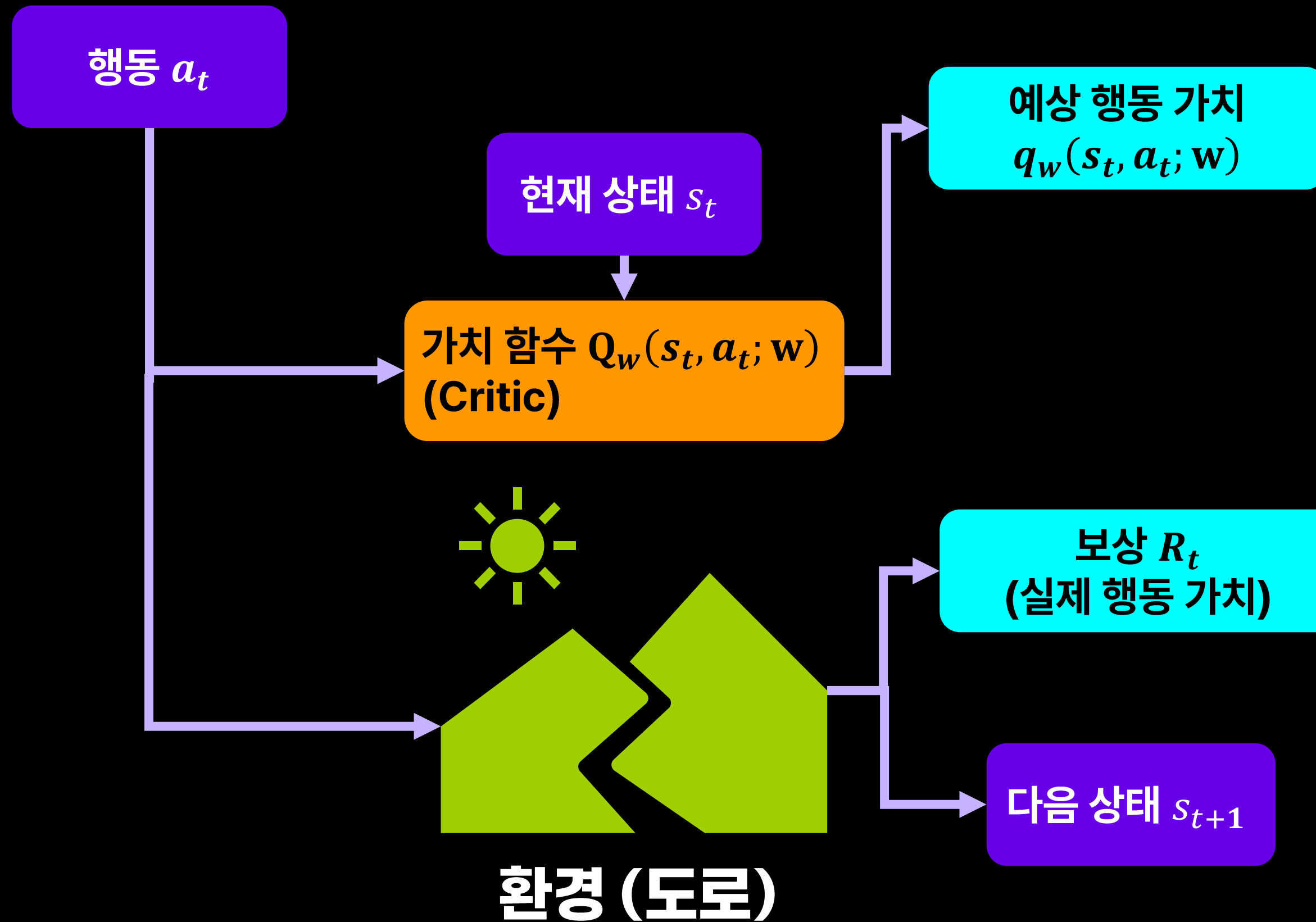
Actor – Critic 학습 과정



5. 가치 함수는 현재 상태와 행동을 입력 받아 Q 함수 값 (예상 행동 가치)을 출력한다.
6. 환경은 행동에 따라 보상을 제공하고, 다음 상태로 넘어간다.
7. 예상 행동 가치를 바탕으로 정책 함수를 업데이트 한다.
8. 예상 행동 가치와 보상 값의 차이를 기반으로 가치 함수를 업데이트 한다.



Actor – Critic 학습 과정

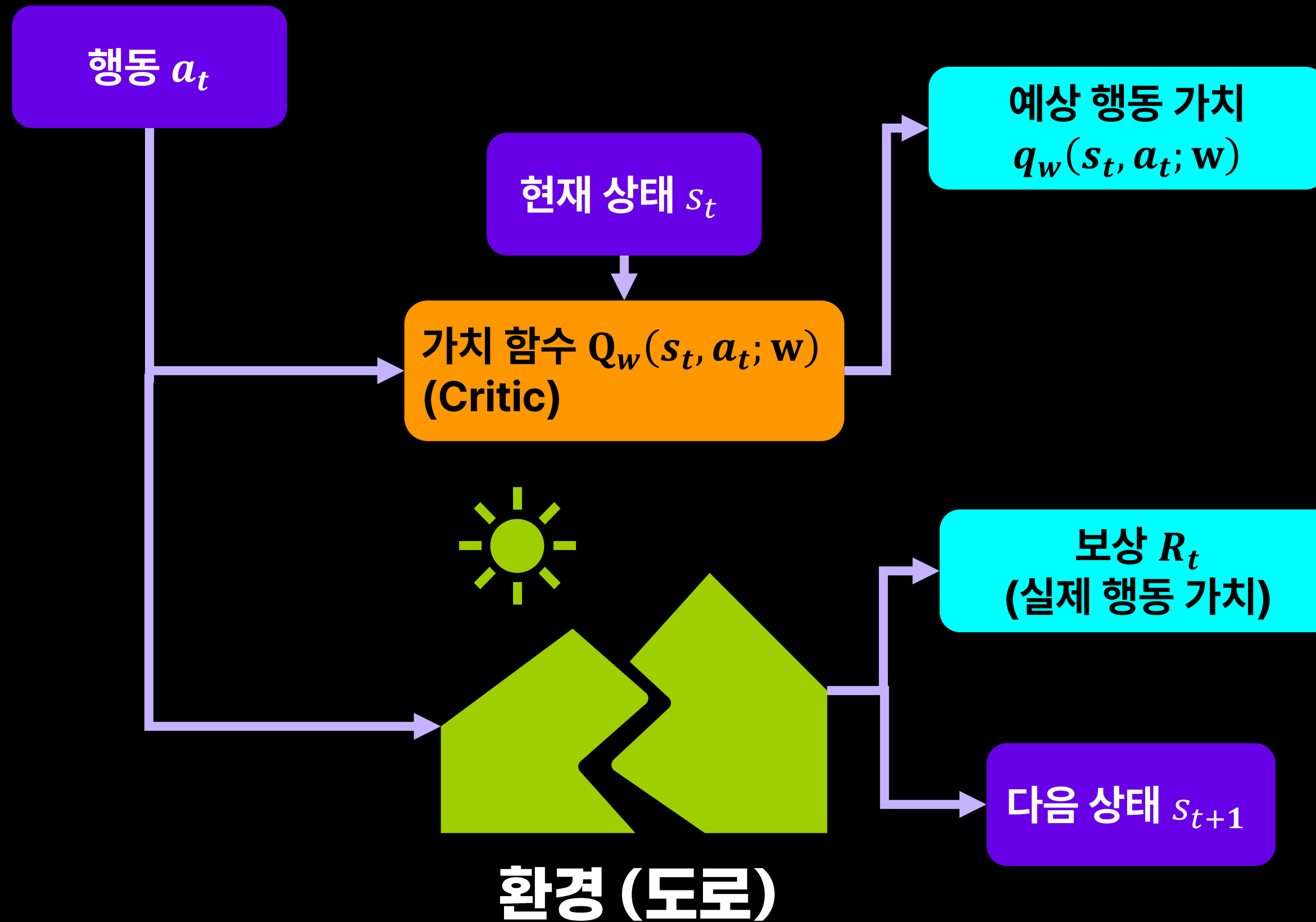


5. 가치 함수는 현재 상태와 행동을 입력 받아 Q 함수 값 (예상 행동 가치)을 출력한다.
6. 환경은 행동에 따라 보상을 제공하고, 다음 상태로 넘어간다.
7. 예상 행동 가치를 바탕으로 정책 함수를 업데이트 한다.
8. 예상 행동 가치와 보상 값의 차이를 기반으로 가치 함수를 업데이트 한다.

 정책 함수와 가치 함수를 업데이트 하는 수식은 [보조자료 2] 를 참조해주세요.



Actor – Critic 학습 과정



5. 가치 함수는 현재 상태와 행동을 입력 받아 Q 함수 값 (예상 행동 가치)을 출력한다.
6. 환경은 행동에 따라 보상을 제공하고, 다음 상태로 넘어간다.
7. 예상 행동 가치를 바탕으로 정책 함수를 업데이트 한다.
8. 예상 행동 가치와 보상 값의 차이를 기반으로 가치 함수를 업데이트 한다.

📄 정책 함수와 가치 함수를 업데이트 하는 수식은 [보조자료 2] 를 참조해주세요.

